



머신 러닝을 이용한 예측 분석

곽재우
노준래
심태양
이은호
이학주





목차

01 프로젝트 소개

02 데이터 이해와 전처리

03 모델 선택과 학습

04 성능 평가와 결과 분석

05 향후 계획 및 개선 방향

06 QnA



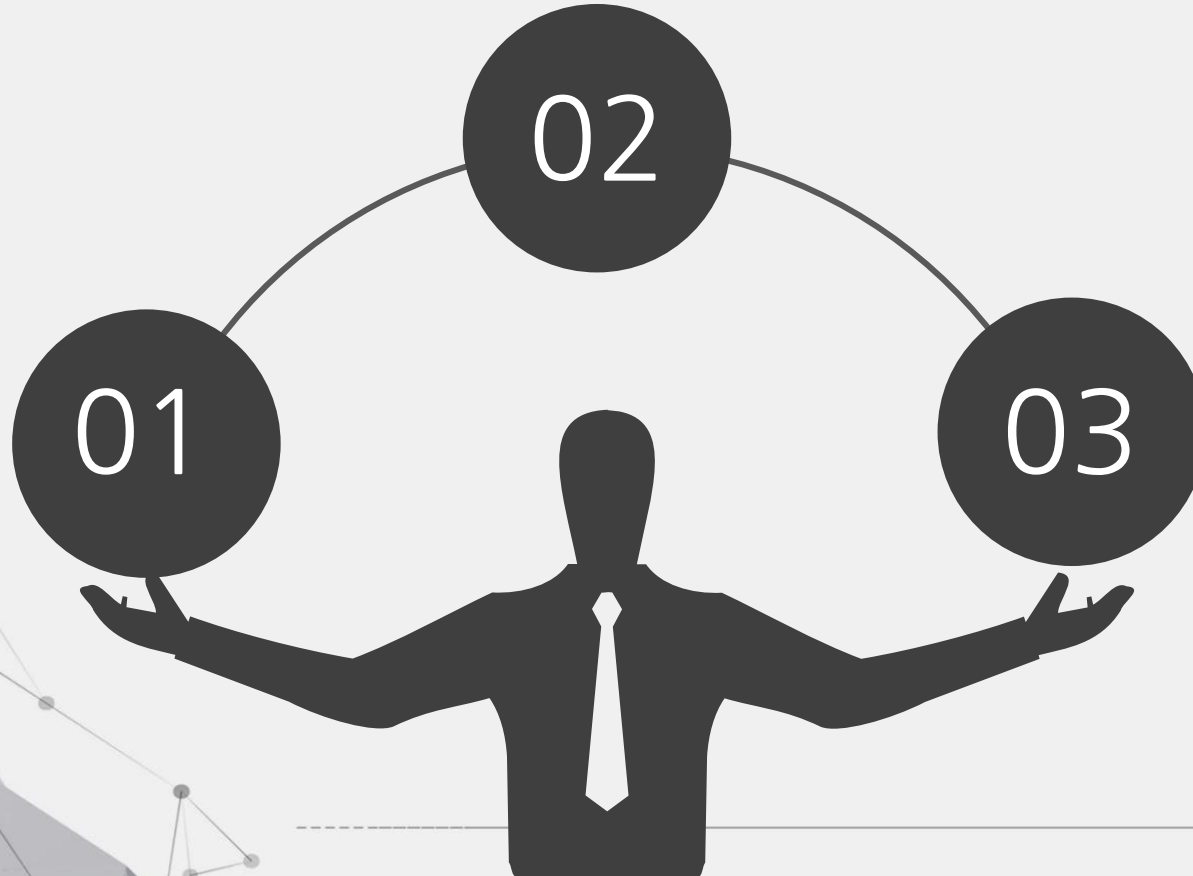


01

프로젝트 소개

프로젝트 목적

캐글 대회(분류1, 회귀1)에서
리더보드 상위 **10%** 진입



데이터 분석 및 머신러닝
대회 출전하여 높은 성적 획득

프로젝트를 진행하며 분석
능력 배양

▲▲▲ 캐글 대회 소개

분류 : 생체신호를 이용한 흡연자 상태의 이진
예측

평가 지표 : ROC AUC SCORE



▲▲▲ 캐글 대회 소개

회귀 : 전복 데이터 세트를 사용한 회귀

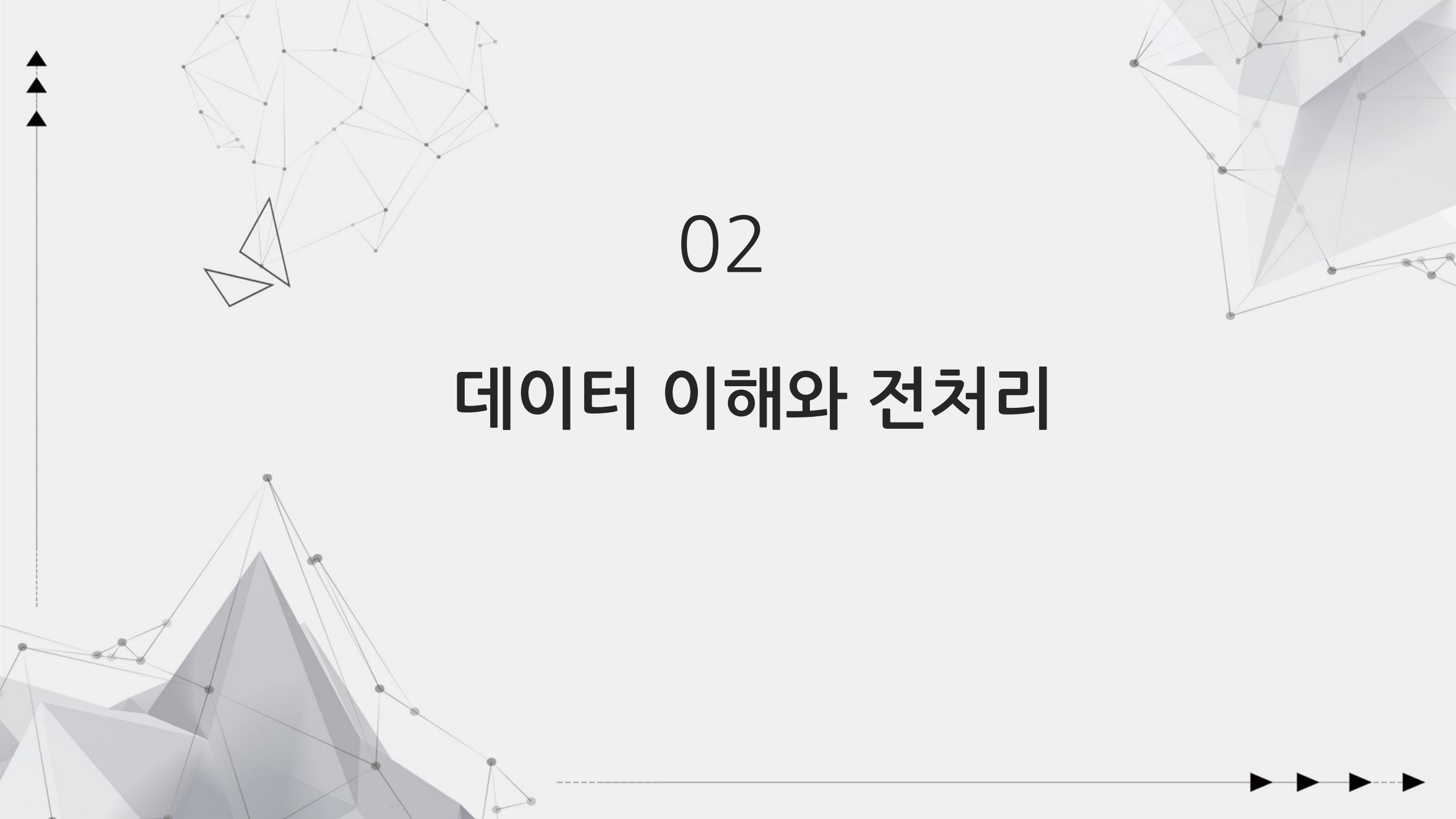
평가 지표 : Root Mean Squared Logarithmic Error





분류

생체 신호를 이용한 흡연자 상태의 이진 예측



02

데이터 이해와 전처리

▲▲▲ 캐글 원본 데이터 및 추가 데이터

원본

train	: 훈련 데이터 세트,	(159256, 24)
test	: 테스트 데이터 세트	(106171, 23)
submission	: 샘플 제출 파일	(106171, 23)

추가

train_dataset : 추가적인 훈련 데이터 세트, (38984, 23)

데이터의 특징 분석

	feature	데이터 타입	결측값	고유타값	max	min
0	age	int64	0	18	85.0	20.0
1	height(cm)	int64	0	15	190.0	130.0
2	weight(kg)	int64	0	29	135.0	30.0
3	waist(cm)	float64	0	548	129.0	51.0
4	eyesight(left)	float64	0	20	9.9	0.1
5	eyesight(right)	float64	0	18	9.9	0.1
6	hearing(left)	int64	0	2	2.0	1.0
7	hearing(right)	int64	0	2	2.0	1.0
8	systolic	int64	0	128	233.0	71.0
9	relaxation	int64	0	94	146.0	40.0
10	fasting blood sugar	int64	0	259	423.0	46.0
11	Cholesterol	int64	0	279	445.0	55.0
12	triglyceride	int64	0	393	999.0	8.0
13	HDL	int64	0	123	359.0	4.0
14	LDL	int64	0	286	1860.0	1.0
15	hemoglobin	float64	0	144	21.1	4.9
16	Urine protein	int64	0	6	6.0	1.0
17	serum creatinine	float64	0	34	11.6	0.1
18	AST	int64	0	196	1090.0	6.0
19	ALT	int64	0	230	2914.0	1.0
20	Gtp	int64	0	444	999.0	2.0
21	dental caries	int64	0	2	1.0	0.0
22	smoking	int64	0	2	1.0	0.0

● 총 22개의 피쳐 변수와 1개의 타겟 변수로 구성

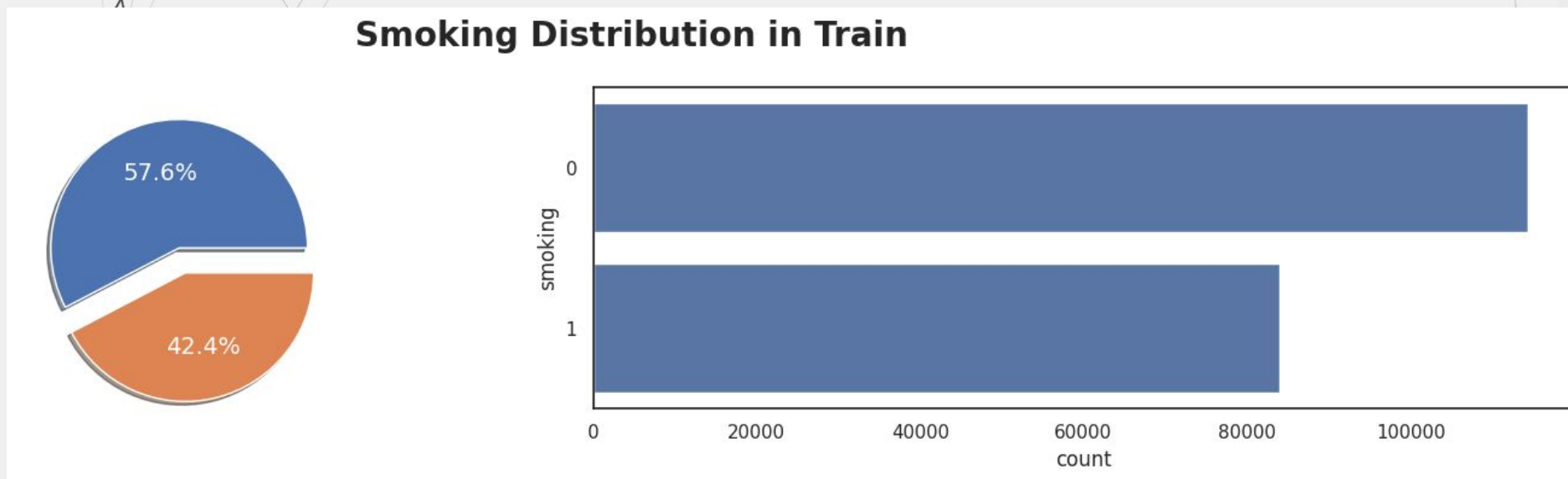
● 타겟 변수 → smoking(흡연 여부)

0 : 비흡연
1 : 흡연



EDA

Target(smoking) 분포 확인



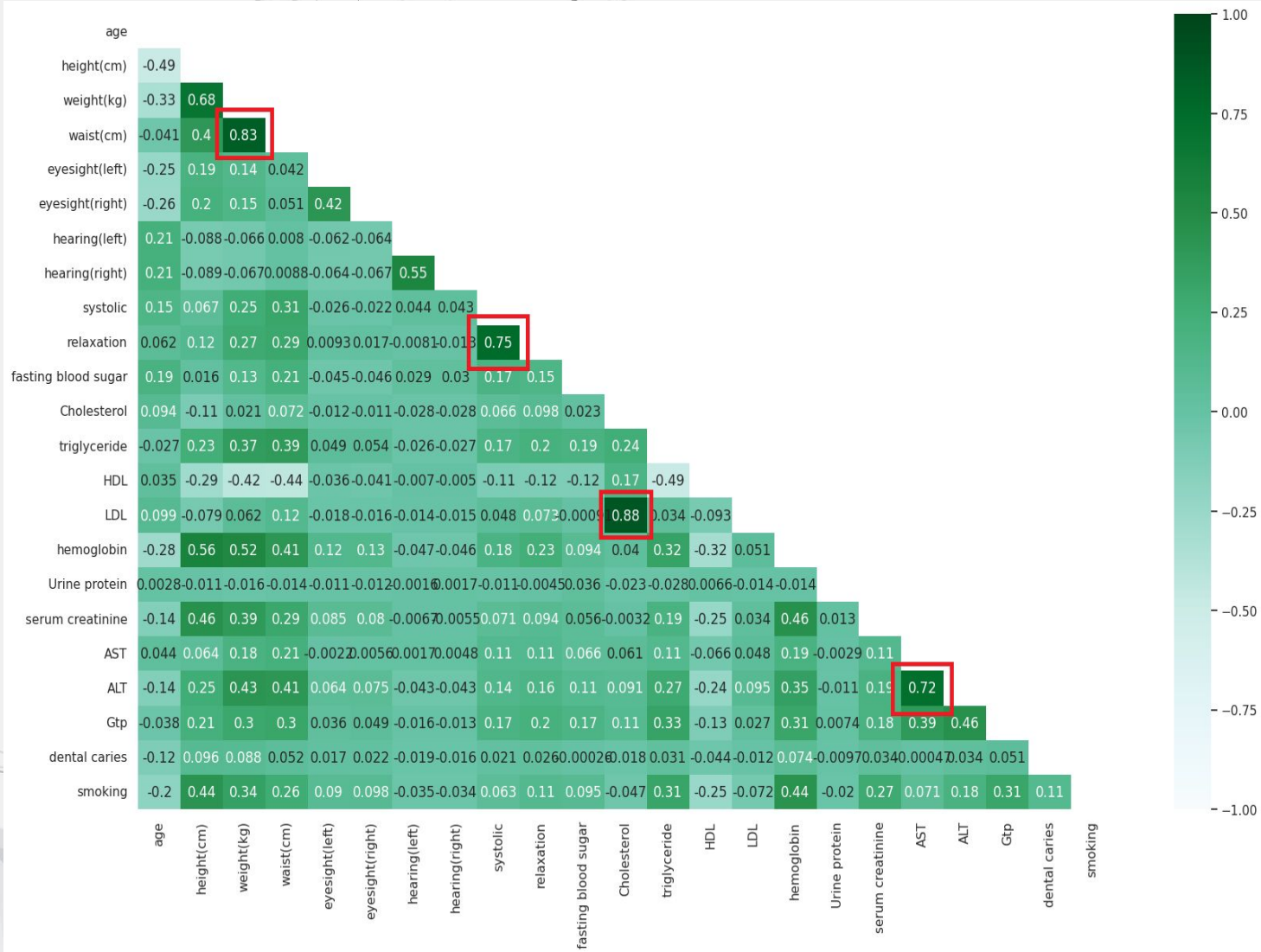
- 비흡연자 57.6% / 흡연자 42.4%

- 균형적인 타겟의 분포



EDA

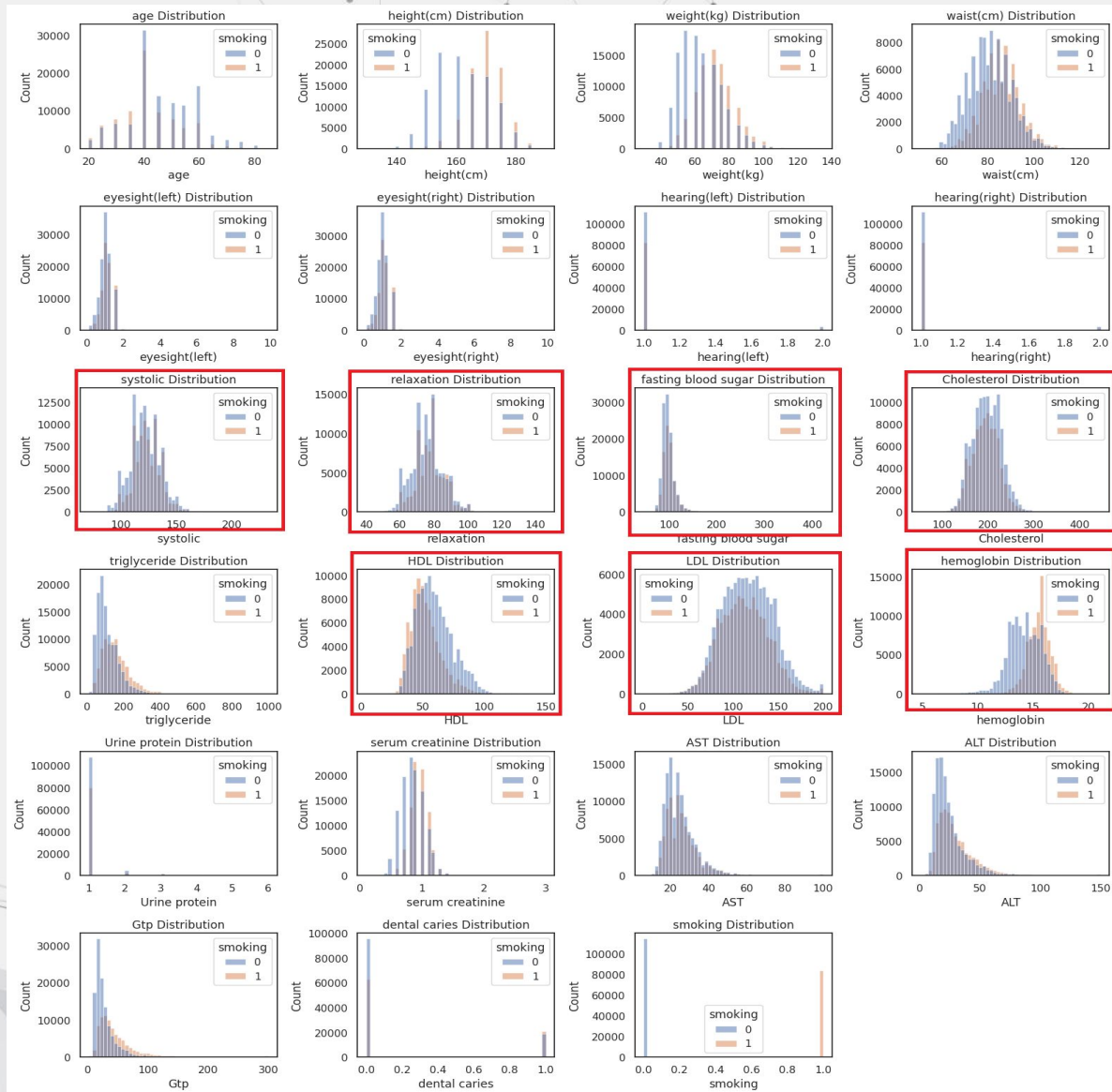
변수 간의 상관관계



- 각 피쳐들 간 상관 관계가 높은지 낮은지 확인
- 키 / 몸무게 / 허리둘레 피쳐가 강한 상관관계 확인
- 혈압과 콜레스테롤 피쳐가 강한 상관관계 확인
- 간 수치에 관련한 피쳐들이 서로 강한 상관관계 확인

EDA

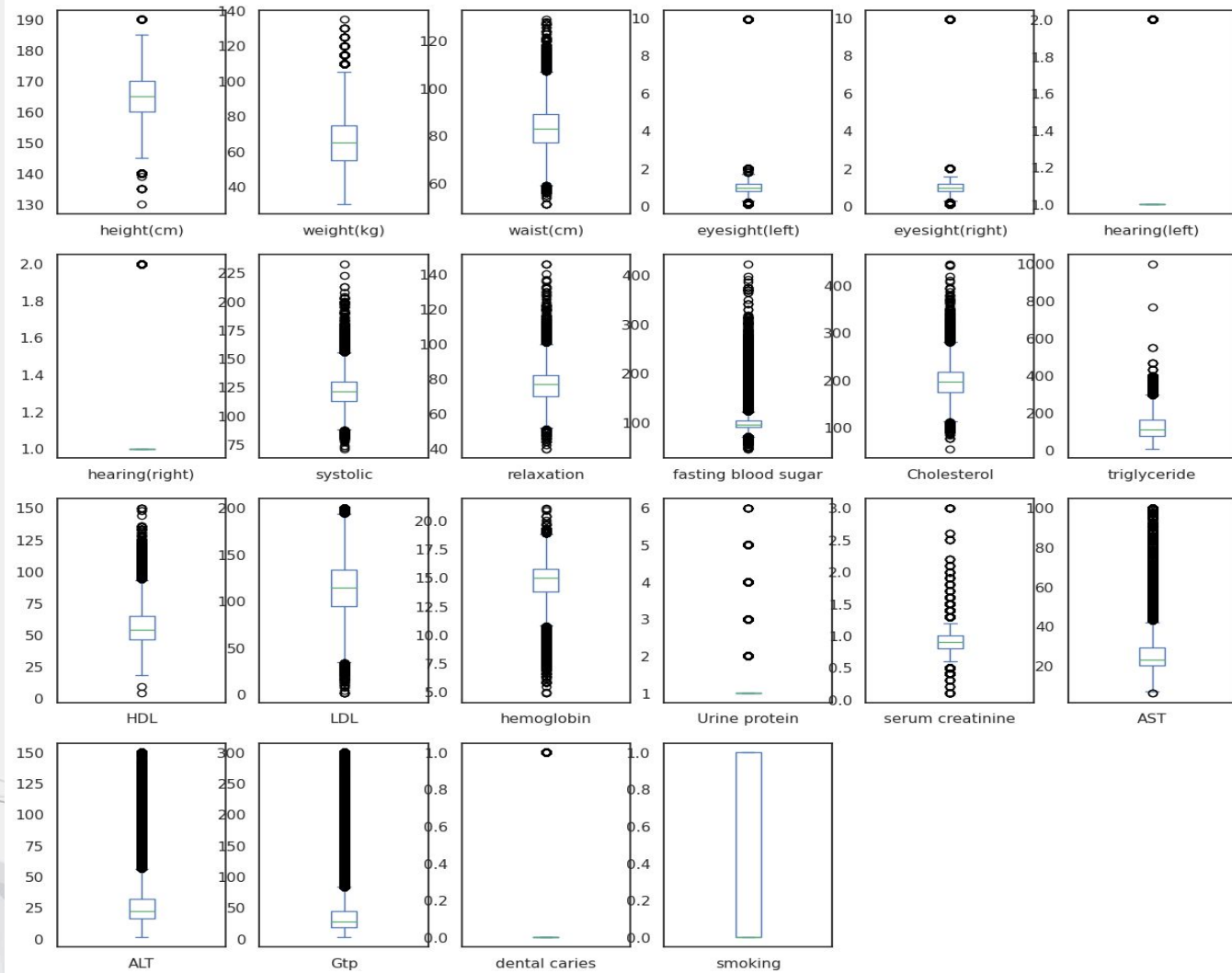
각 데이터 분포도



■ 혈액 관련 피처들이 정규분포의 형태를 띄고 있는 것을 확인

EDA

각 데이터의 이상치

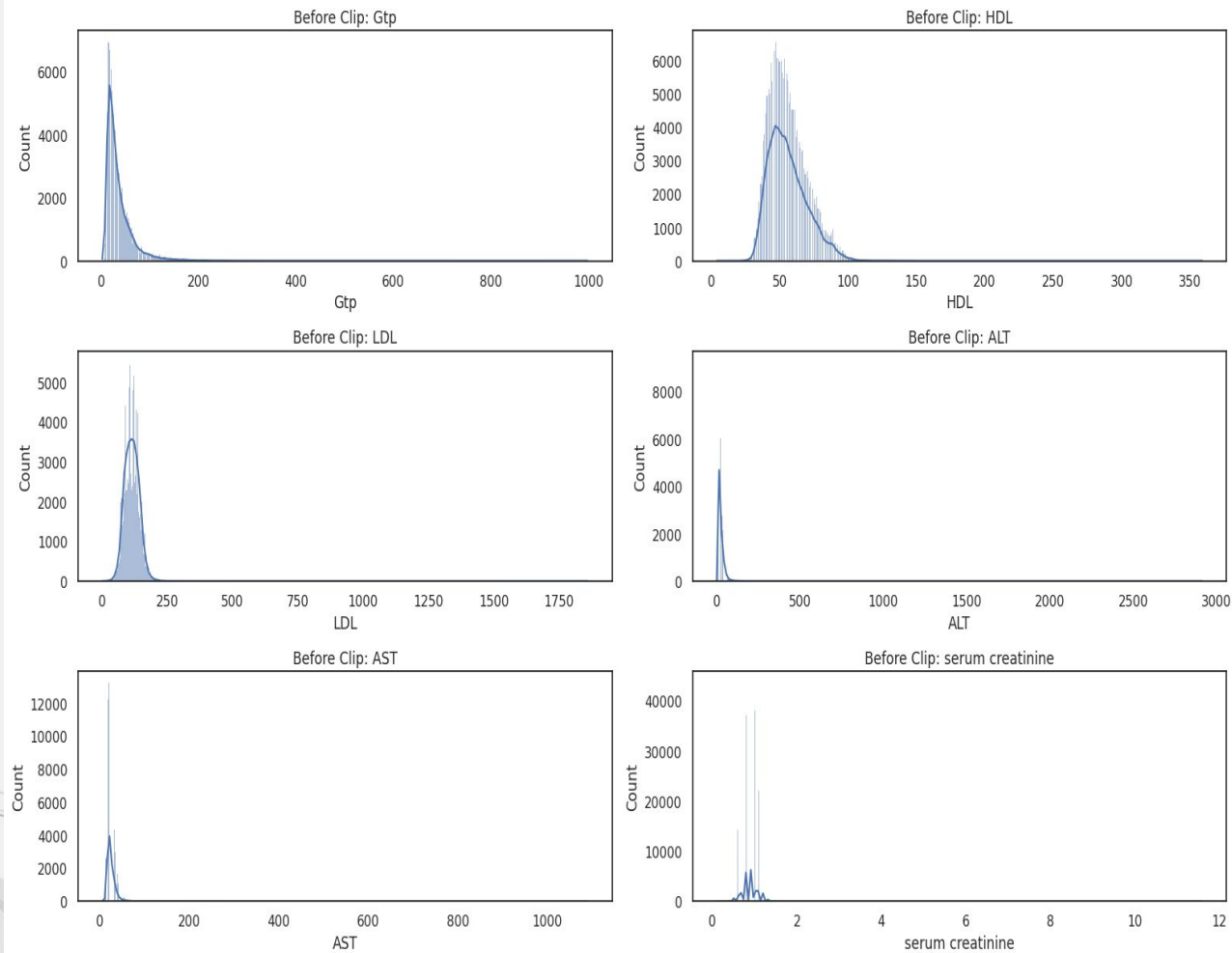


- IQR 기준 이상치 확인

- 도메인적 지식을 바탕으로 이상치 제거 여부 보류

EDA

특정 피처에 대한 임계값 조정 전



-오른쪽으로 긴 꼬리를 형성

- 대체적인 값들이 정규분포를 따르는 모습을 보이고 있으나
이상치를 가진 값들이 존재



EDA

피쳐 엔지니어링

```
1 # train 데이터에 대한
2 train['Gtp'] = train['Gtp'].clip(lower = 0, upper = 300)
3 train['HDL'] = train['HDL'].clip(lower = 0, upper = 150)
4 train['LDL'] = train['LDL'].clip(lower = 0, upper = 200)
5 train['ALT'] = train['ALT'].clip(lower = 0, upper = 150)
6 train['AST'] = train['AST'].clip(lower = 0, upper = 100)
7 train['serum creatinine'] = train['serum creatinine'].clip(lower = 0, upper = 3)
8
9 # test 데이터에 대한
10 test['Gtp'] = test['Gtp'].clip(lower = 0, upper = 300)
11 test['HDL'] = test['HDL'].clip(lower = 0, upper = 150)
12 test['LDL'] = test['LDL'].clip(lower = 0, upper = 200)
13 test['ALT'] = test['ALT'].clip(lower = 0, upper = 150)
14 test['AST'] = test['AST'].clip(lower = 0, upper = 100)
15 test['serum creatinine'] = test['serum creatinine'].clip(lower = 0, upper = 3)
```

train, test데이터의 피쳐['Gtp', 'HDL', 'LDL', 'ALT', 'AST', 'serum creatinine']들의 값이 한 쪽으로 몰려있는 이상치 확인

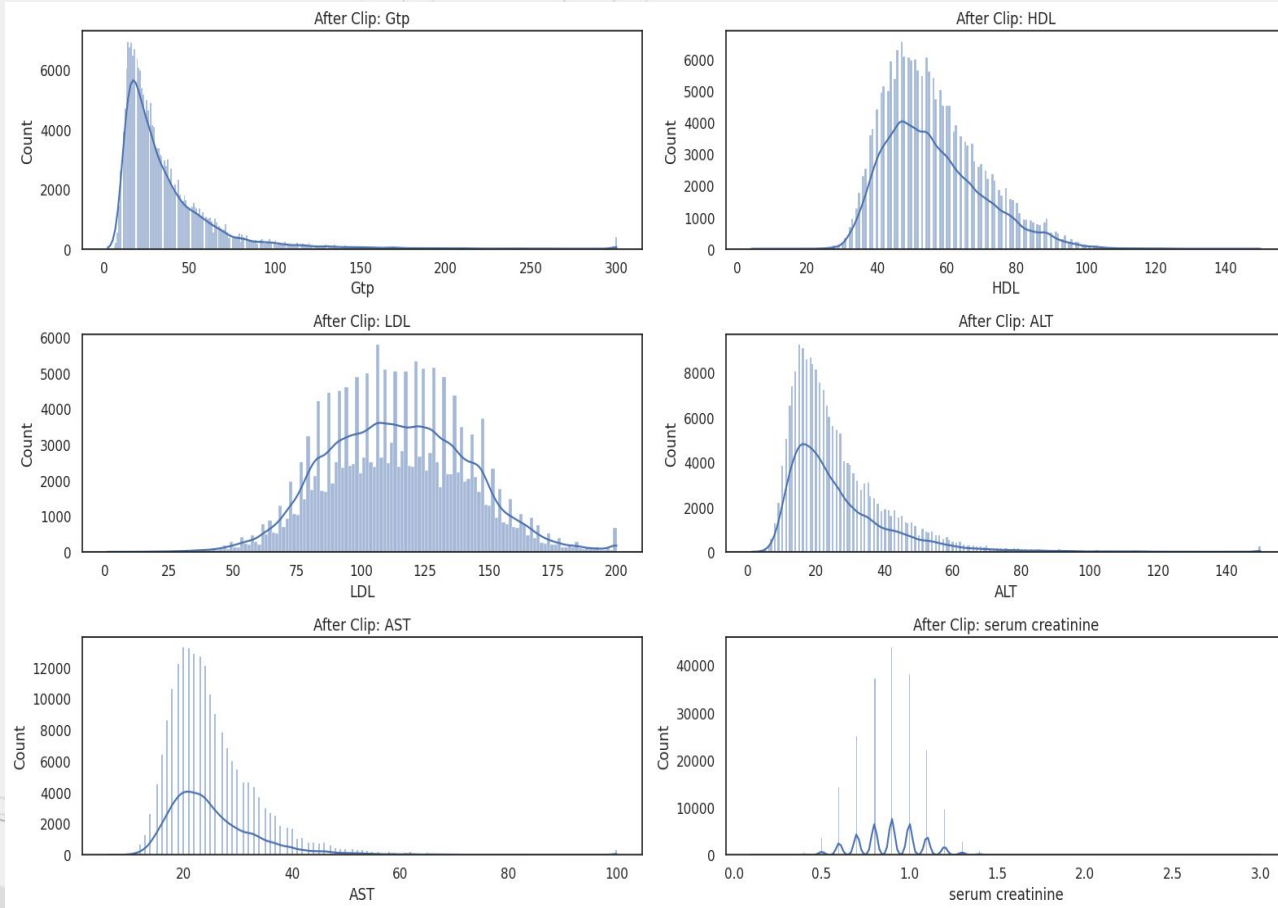


clip() 함수 사용하여 임계값 조정 (300, 150, 200, 150, 100, 3)



EDA

특정 피쳐에 대한 임계값 조정 후



- 이상치를 임계값으로 조정 (300, 150, 200, 150, 100, 3)



전처리

기존 data와 수집한 data 병합

```
1 train = pd.concat([ltrain, train_dataset])  
2 train
```

	age	height(cm)	weight(kg)	waist(cm)	eyesight(left)	eyesight(right)	hearing(left)	hearing(right)	systolic	relaxation	...	HDL	LDL	hemoglobin	Urine protein	serum creatinine	AST
0	55	165	60	81.0	0.5	0.6	1	1	135	87	...	40	75	16.5	1	1.0	22
1	70	165	65	89.0	0.6	0.7	2	2	146	83	...	57	126	16.2	1	1.1	27
2	20	170	75	81.0	0.4	0.5	1	1	118	75	...	45	93	17.4	1	0.8	27
3	35	180	95	105.0	1.5	1.2	1	1	131	88	...	38	102	15.9	1	1.0	20
4	30	165	60	80.5	1.5	1.0	1	1	121	76	...	44	93	15.4	1	0.8	19
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
38979	40	165	60	80.0	0.4	0.6	1	1	107	60	...	61	72	12.3	1	0.5	18
38980	45	155	55	75.0	1.5	1.2	1	1	126	72	...	76	131	12.5	2	0.6	23
38981	40	170	105	124.0	0.6	0.5	1	1	141	85	...	48	138	17.1	1	0.8	24
38982	40	160	55	75.0	1.5	1.5	1	1	95	69	...	79	116	12.0	1	0.6	24
38983	55	175	60	81.1	1.0	1.0	1	1	114	66	...	64	137	13.9	1	1.0	18

198240 rows x 23 columns

- 병합 전 **train.shape = (159256, 23)** original.shape = (38984, 23)

- 병합 후 **train.shape = (198240, 23)**



▲ 전처리

▲ 중복 데이터 제거 및 결측값 확인

```
1 train_df.shape
```

```
(198240, 23)
```

```
1 train_df.drop_duplicates().shape
```

```
(192723, 23)
```

- 총 5517개 제거

- 최종 전처리 후 train.shape = (192723, 23)

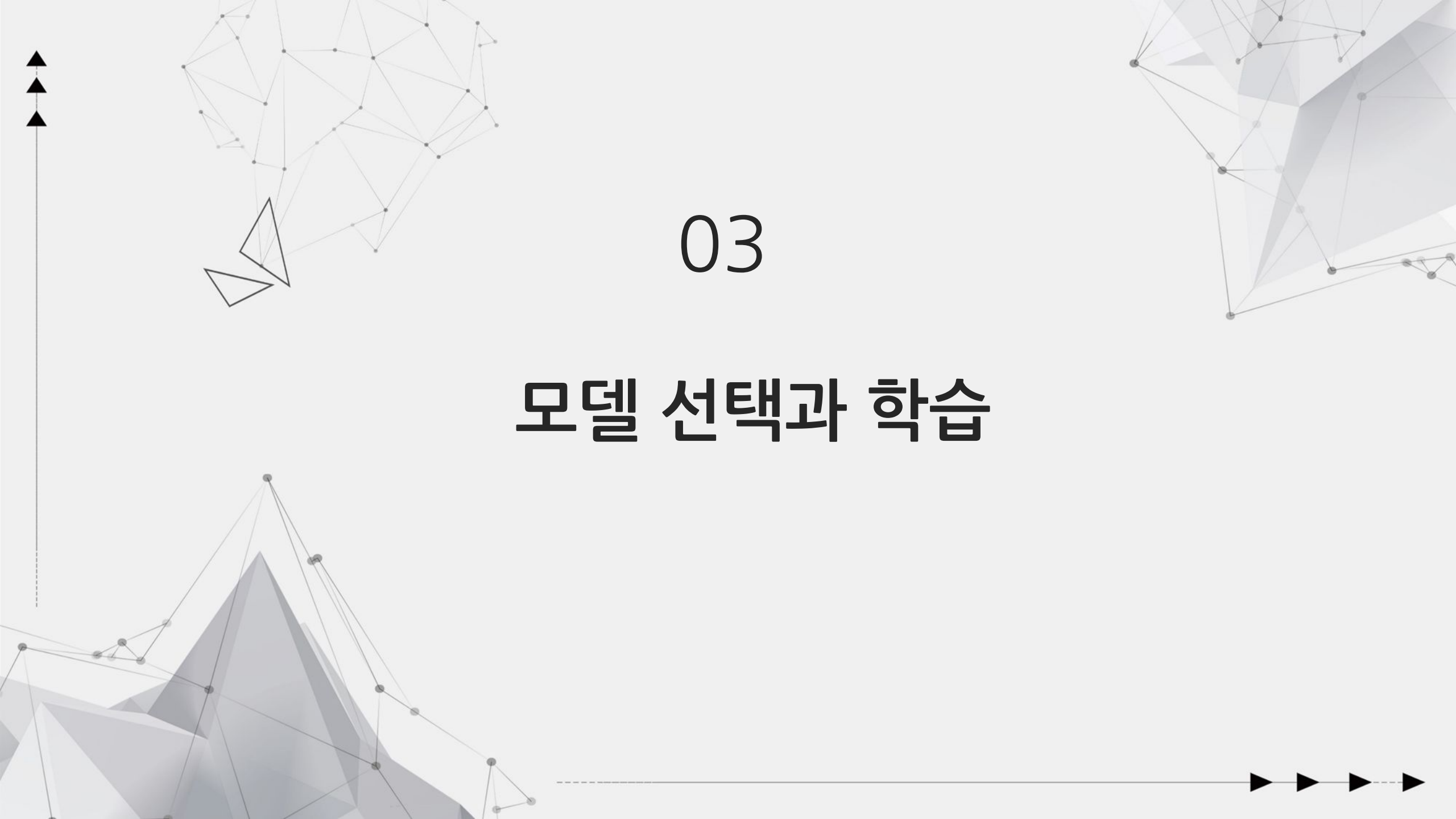
```
1 # 누락된 값 확인  
2 train.isnull().sum()
```

```
age 0  
height(cm) 0  
weight(kg) 0  
waist(cm) 0  
eyesight(left) 0  
eyesight(right) 0  
hearing(left) 0  
hearing(right) 0  
systolic 0  
relaxation 0  
fasting blood sugar 0  
Cholesterol 0  
triglyceride 0  
HDL 0  
LDL 0  
hemoglobin 0  
Urine protein 0  
serum creatinine 0  
AST 0  
ALT 0  
Gtp 0  
dental caries 0  
smoking 0  
dtype: int64
```

```
1 # 누락된 값 확인  
2 test.isnull().sum()
```

```
age 0  
height(cm) 0  
weight(kg) 0  
waist(cm) 0  
eyesight(left) 0  
eyesight(right) 0  
hearing(left) 0  
hearing(right) 0  
systolic 0  
relaxation 0  
fasting blood sugar 0  
Cholesterol 0  
triglyceride 0  
HDL 0  
LDL 0  
hemoglobin 0  
Urine protein 0  
serum creatinine 0  
AST 0  
ALT 0  
Gtp 0  
dental caries 0  
dtype: int64
```





03

모델 선택과 학습

모델 선택 및 계획

교차검증

- Fold의 개수를 조정

하이퍼 파라미터 조정

- 그리드서치 cv
- optuna
- 직접 하이퍼 파라미터 수정
- kaggle discussion 참고

다양한 모델 및 기법 사용

- LightGBM
- Catboost
- XGBoost
- 앙상블

피처 엔지니어링

- 특정 피처에 대한 임계값을 설정
- 수치형 피처에 대한 스케일링
- 범주형 피처에 대한 인코딩
- 파생피처 생성

01

02

03

04



모델 선택

(0.86803) submission_5Fold_depth(12)
(0.86907) submission_stacker_robust
(0.87041) submission_stacker_standard
(0.87050) submission_stacker_minmax
(0.87194) submission_5Fold(2)(임계값 -50)
(0.87386) submission_5Fold(X)
(0.87413) submission_5Fold(3)(임계값 +50)
(0.87413) submission_5Fold
(0.87474) submission_LGB
(0.87553) submission_LGB_핫코딩_minmax(rate=0.3)
(0.87626) submission_LGB(X)
(0.87760) submission_LGB_핫코딩_minmax(rate=0.12)
(0.87767) submission_LGB_핫코딩
(0.87770) submission_LGB_핫코딩_minmax(boosting=gdbt)
(0.87784) submission_LGB_fold_원핫인코딩
(0.87795) lgb_submission_minmax
(0.87797) lgb_submission_minmax
(0.87802) lgb_submission_minmax

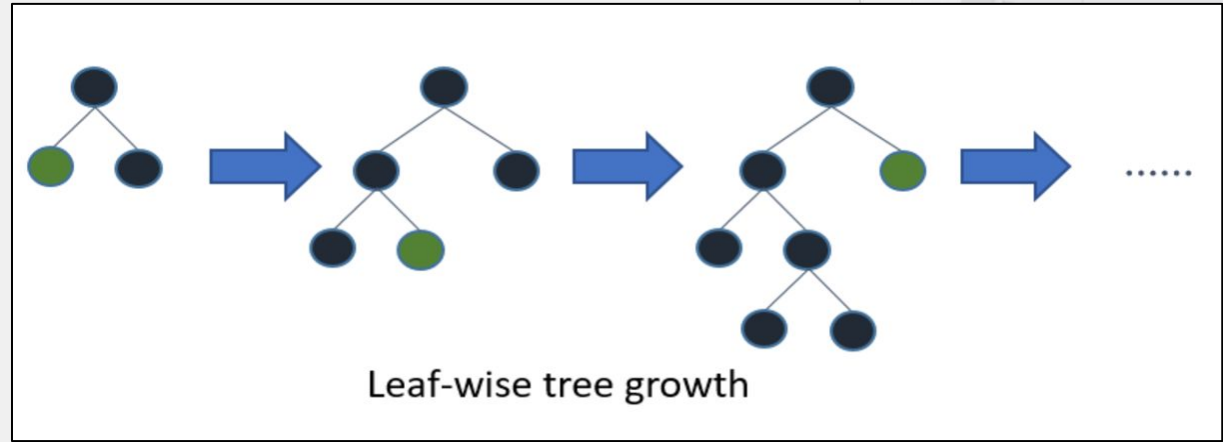
성능 향상을 위한 여러가지 방법을 사용

- 앙상블
- 스택킹

LightGBM 모델이 제일 좋은 성능을 보이고
현재 데이터에 적합하다고 판단



- 그래디언트 부스팅
- 여러개의 약한 학습기를 결합하여 강력한 학습기를 만드는 기법
- 대용량 데이터셋에 대한 빠른 속도와 높은 성능



- 리프 중심 트리 분할
- 효율적인 메모리 사용
- 다양한 손실 함수 지원

모델 학습

Input Data

type	feature	count
categorical	hearing(left), hearing(right), Urine protein, dental caries	4
continuous	age, height(cm), weight(kg), waist(cm), eyesight(left), eyesight(right), systolic, relaxation, fasting blood sugar, cholesterol, triglyceride, HDL, LDL, hemoglobin, serum creatinine, AST, ALT, Gtp	18

- 기존 피쳐만 이용해서 학습에 사용
- **GridSearchCV / optuna / kaggle discussion**
활용하여 현재 모델에 최적의 파라미터를 발견

Parameter

objective	"binary"
lambda_l1	3.1422772805786794
lambda_l2	0.3912821668022086
boosting_type	"gbdt"
num_leaves	120
feature_fraction	0.24955364718656628
bagging_fraction	0.9712078900905778
bagging_freq	9
min_child_samples	200
num_boost_round	600
learning_rate	0.1

※ 파이썬에서 제공하는 모델을 사용하여 파라미터명이 다를 수 있음



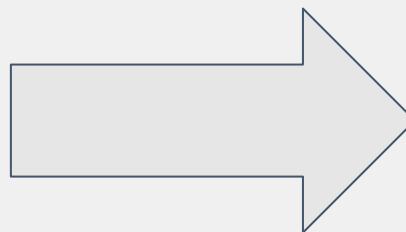
▲ 성능 향상 및 모델 안정성 평가

```
0 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
1 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
2 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
3 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
4 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
5 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235816854990584 0.4235875706214689
6 (169920, 22) (28320, 22) (169920,) (28320,) 0.4235875706214689 0.4235522598870056
```

- 모델의 안정성 및 성능 향상을 위해 **Stratified Fold**
- 동일한 기준으로 **split** 하기 위해 각 **Fold**의 비율 점검

```
데이터:0, 난수 값: 1, auc: 0.8787623982559245
데이터:0, 난수 값: 3, auc: 0.8786765195520945
데이터:0, 난수 값: 5, auc: 0.8790705085963884
데이터:0, 난수 값: 7, auc: 0.8792501762206286
데이터:0, 난수 값: 1024, auc: 0.8794633588554676
데이터:0, 난수 값: 9999, auc: 0.8792320475770777
```

- 모델의 **random_seed**를 변경하여 안정성 평가 진행



```
1 submission["smoking"] = np.mean(preds_list, axis = 0)
2 submission
```

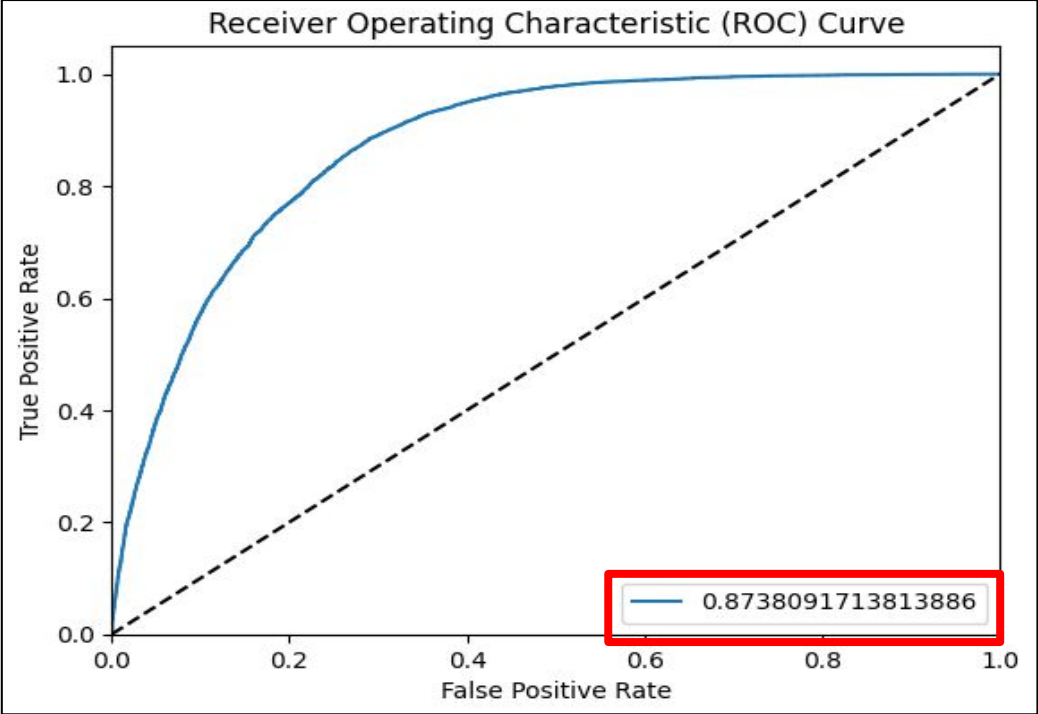
	id	smoking	
0	159256	0.646688	
1	159257	0.213923	
2	159258	0.327293	
3	159259	0.017322	
4	159260	0.667335	
...	...	...	
106166	265422	0.680284	
106167	265423	0.531902	
106168	265424	0.519811	
106169	265425	0.096709	
106170	265426	0.036408	

106171 rows × 2 columns

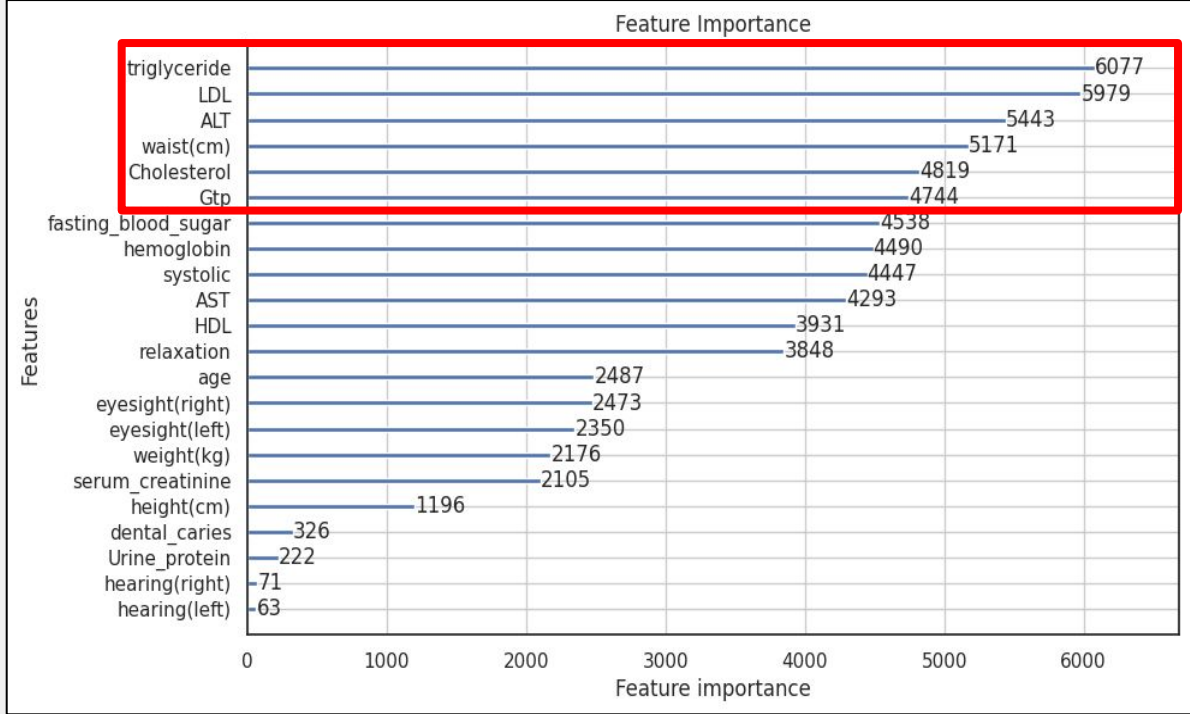
모든 예측값을 **평균** -> 최종 파일 생성



성능 평가와 결과 분석



좋은 성능을 가진 모델이라고 판단



혈액 / 혈관 및 콜레스테롤 수치 상위권 차지

Kaggle 제출시 결과

		lgb_submission_final_8.csv		완료(마감일 이후) · 6일 전		0.87805	
187	▼ 27	산		0.87806	5	5개월	
188	▲ 5	완웨이뉴에		0.87806	5	6개월	
189	▲ 5	에티엔A		0.87806	삼	5개월	
190	▲ 5	슈밤 차반		0.87806	5	5개월	
191	▼ 11	낸시		0.87799	11	6개월	
192	▲ 19	파이살 알스레이드		0.87798	삼	5개월	
193	▲ 7	은행 계좌		0.87797	10	6개월	
194	▲ 7	유티아오		0.87794	24	5개월	

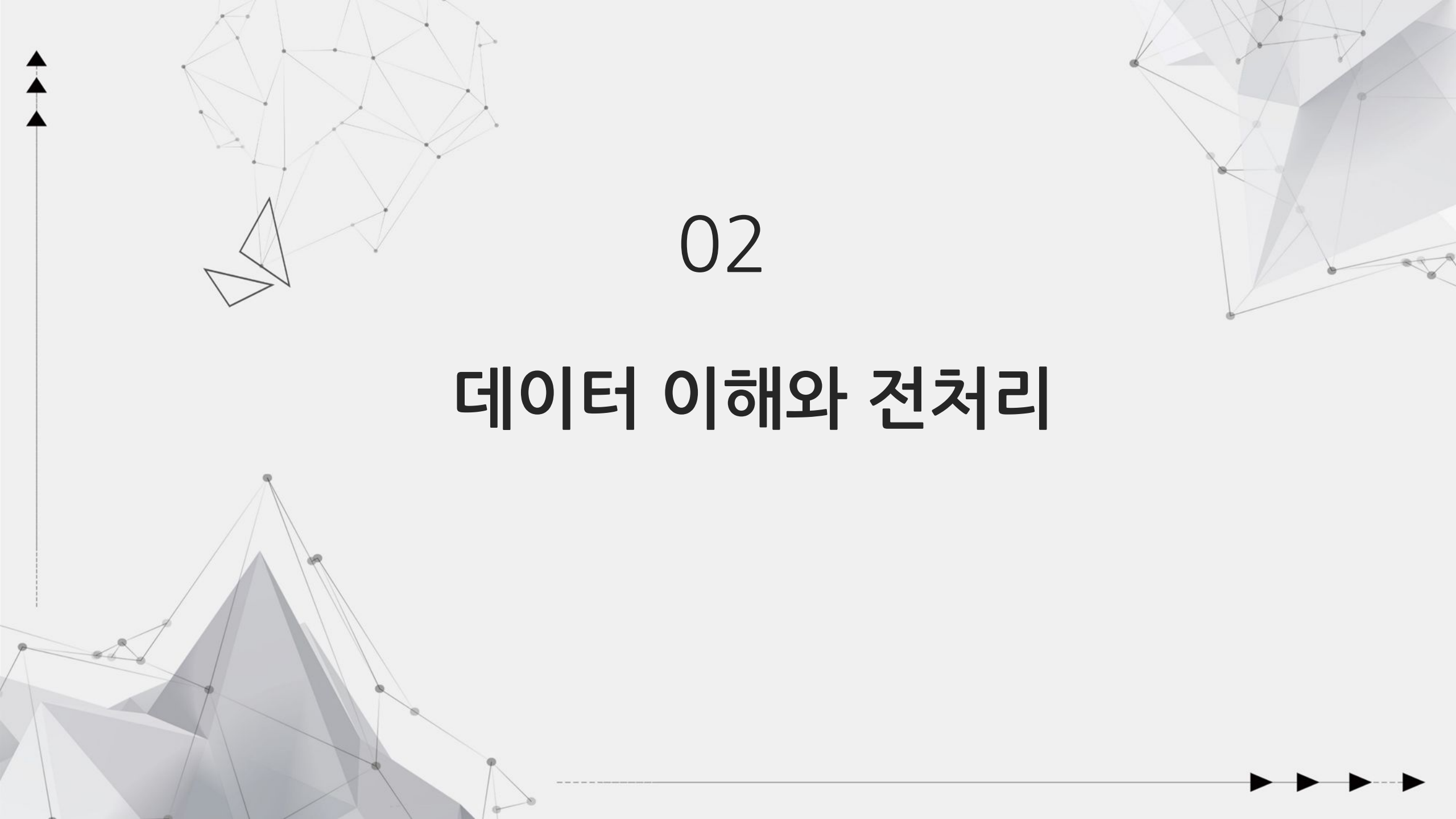
목표 했던 상위 10% 달성





회귀

전복 데이터 세트를 사용한 회귀



02

데이터 이해와 전처리

캐글 원본 데이터 및 추가 데이터

원본

train	: 훈련 데이터 세트,	(90615, 9)
test	: 테스트 데이터 세트	(60411, 8)
submission	: 샘플 제출 파일	(60411, 2)

추가

abalone : 추가적인 훈련 데이터 세트, (4177, 9)



데이터의 특징 분석

	feature	데이터 타입	결측값	고유통	max	min
0	id	int64	0	90615	90614	0
1	Sex	object	0	3	M	F
2	Length	float64	0	157	0.815	0.075
3	Diameter	float64	0	126	0.65	0.055
4	Height	float64	0	90	1.13	0.0
5	Whole weight	float64	0	3175	2.8255	0.002
6	Whole weight.1	float64	0	1799	1.488	0.001
7	Whole weight.2	float64	0	979	0.76	0.0005
8	Shell weight	float64	0	1129	1.005	0.0015
9	Rings	int64	0	28	29	1

- 총 8개의 피쳐 변수와 1개의 타겟 변수로 구성

- 타겟 변수 → **Rings(전복 나이)**

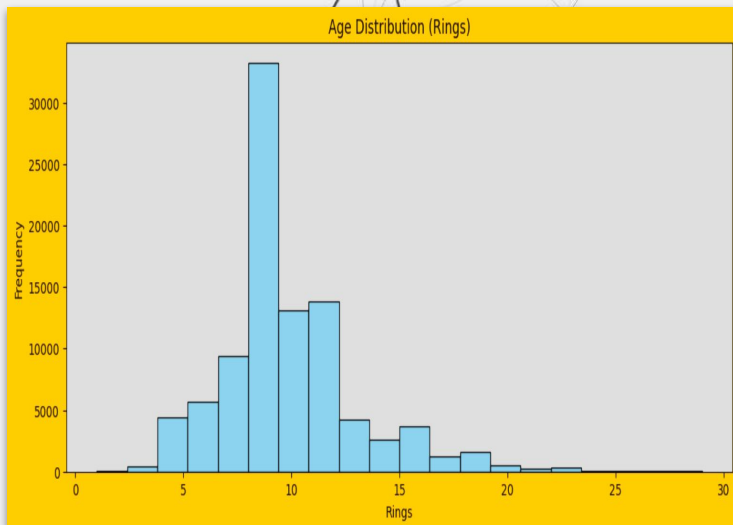
고유통 : 1 ~ 29



EDA

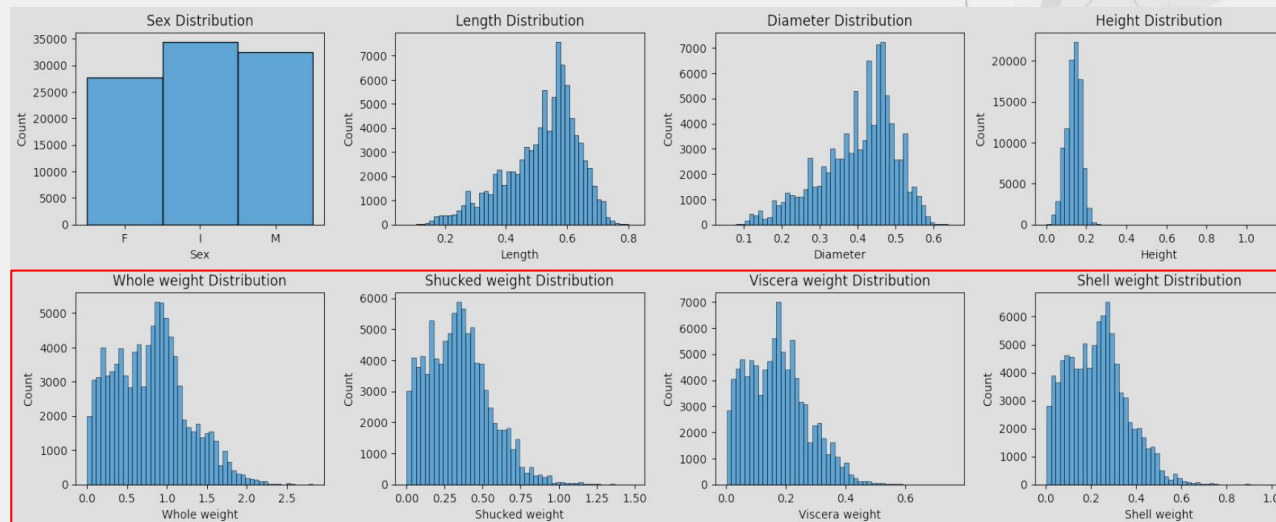
Exploratory Data Analysis

타겟 (Rings) 분포 확인



평균 : 9.71
표준편차 : 3.18
최소값 : 1.0
25% : 8.0
50% : 9.0
75% : 11.0
최대값 : 29.0

각 데이터의 분포 확인



평균이 중앙값인 9.0을 상회하는 9.71로 나타나며
오른쪽으로 긴 꼬리를 형성하는 것을 확인

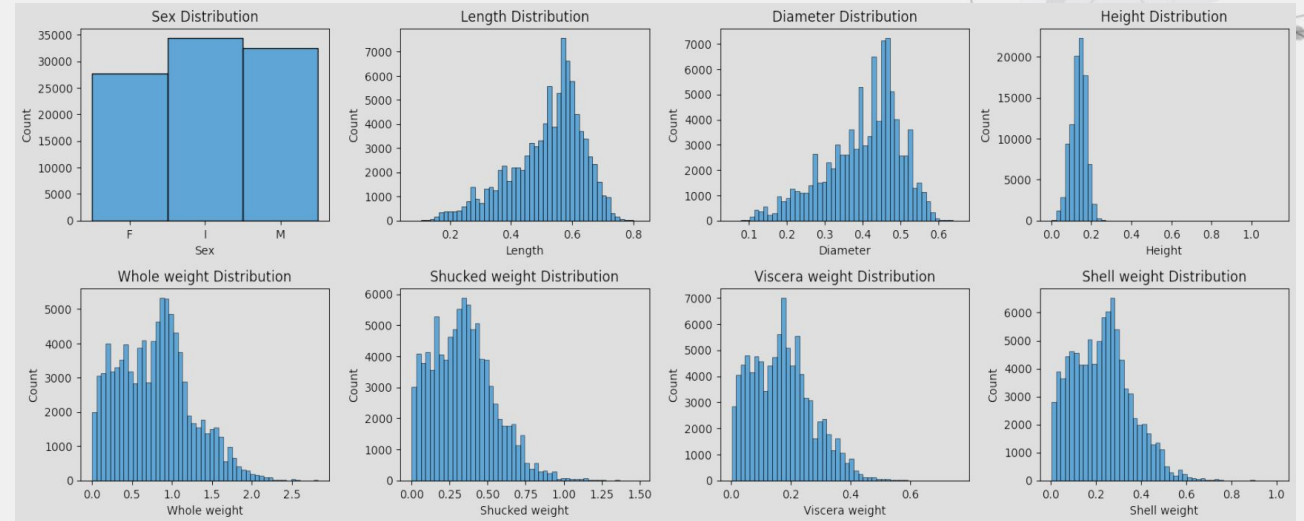
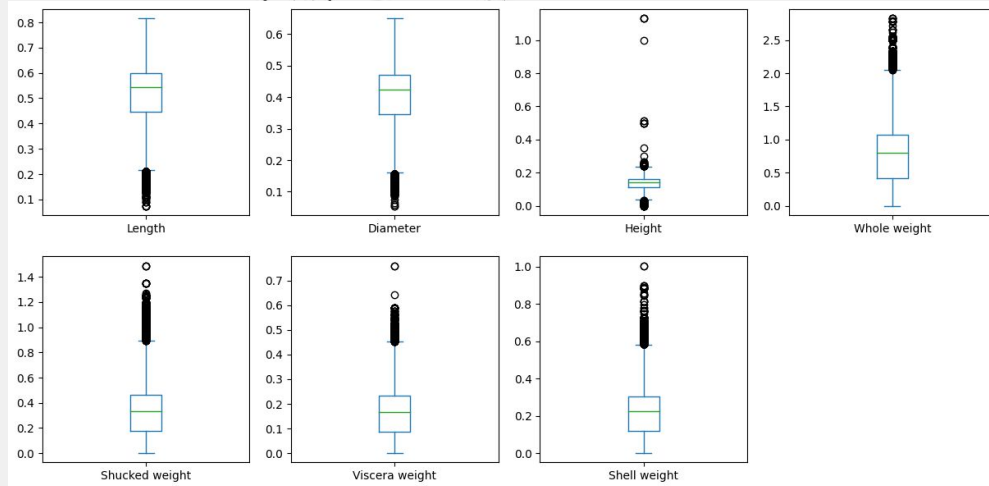
각 데이터의 분포도 범주형 데이터인 성별과 길이, 지름을 제외하고는
오른쪽으로 긴 꼬리를 형성하는 것을 확인



EDA

Exploratory Data Analysis

각 데이터의 이상치 확인

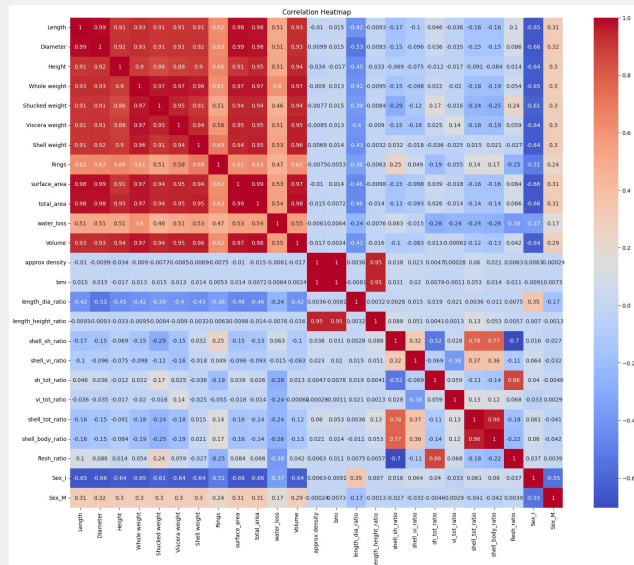
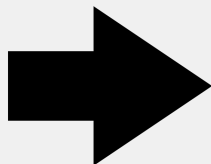
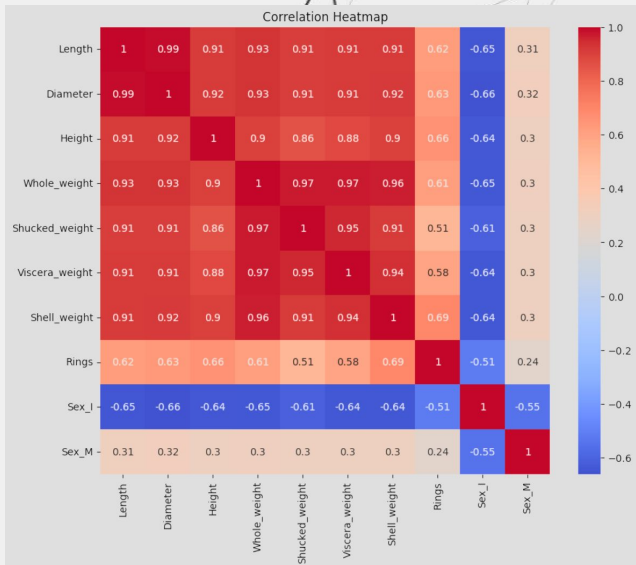


박스 플롯을 이용하여 이상치를 파악한 결과 길이와 지름은 평균보다 낮은 값에서, 나머지는 평균보다 높은 값에서 이상치가 나타나고 있음을 확인

EDA

Exploratory Data Analysis

데이터 상관관계 확인



0.1 이하 : 거의 없음

0.1~0.3 : 약한 상관관계

0.3~0.5 : 보통 상관관계

0.5 이상 : 강한 상관관계

기존 데이터간의 강한 상관관계 확인

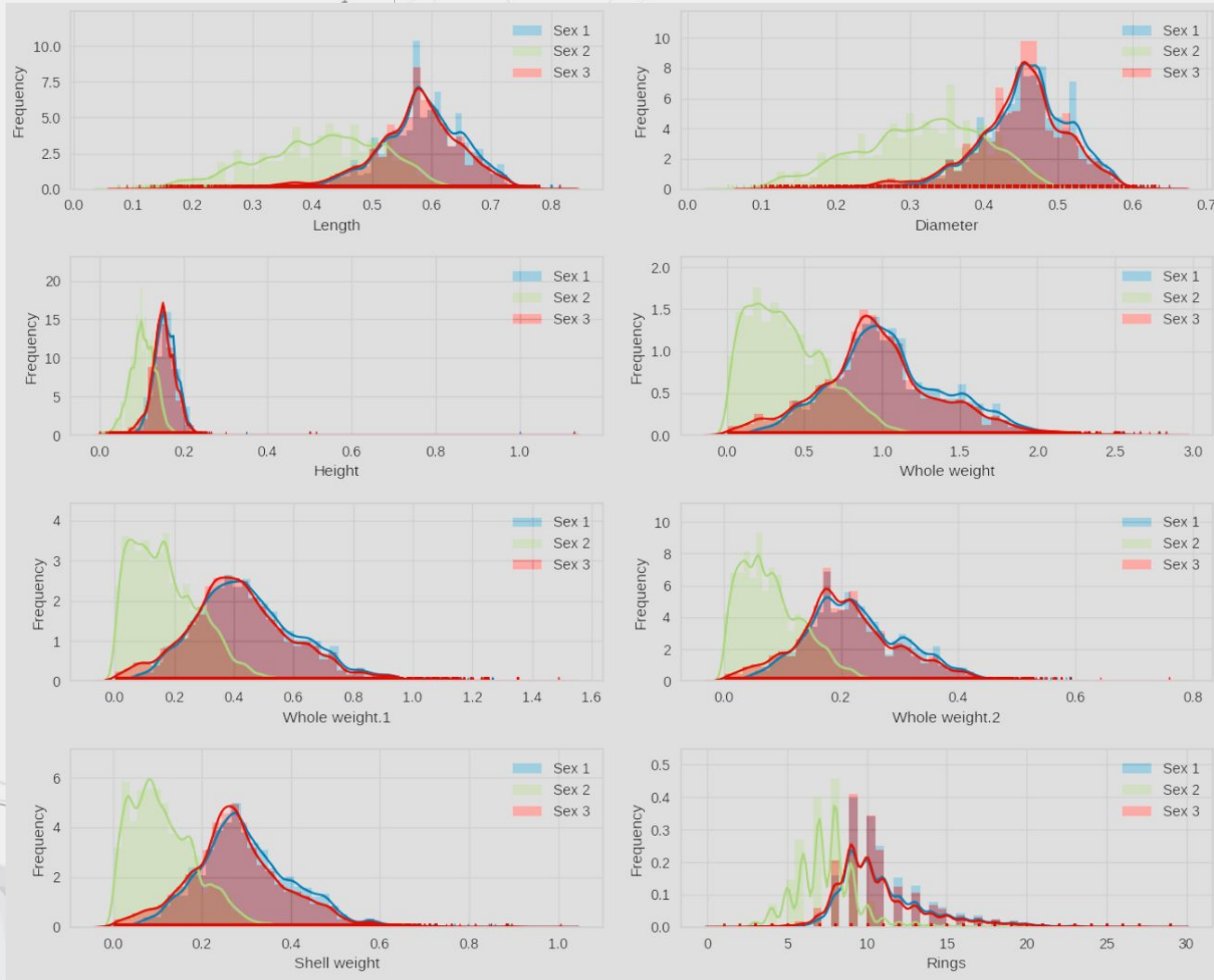
새로운 피쳐 데이터간의 미약한 상관관계 확인

EDA

Exploratory Data Analysis

성별 데이터 분포 확인

1: Female
2: immature
3: male



Female 데이터와 Male 데이터는 비슷한 분포의 양상

Immature 데이터는 홀로 다른 데이터의 분포의 양상



피처엔지니어링 및 전처리

Feature Engineering & preprocessing

추가 데이터 병합을 위한 전처리

```
[ ] 1 # id 삭제
     2 train = train.drop(['id'], axis = 1)
     3 train.columns = original.columns
```

```
[ ] 1 # submission id에 대입하기 위한 변수 test_id(test데이터에서 id데이터 삭제하기 전에 미리 저장)
     2 test_id = test['id']
     3 # test데이터에서 id 삭제
     4 test = test.drop('id', axis = 1)
     5 # test데이터의 컬럼들을 train데이터의 컬럼과 동일하게
     6 test.columns = train.columns
     7 test.head()
```

	Sex	Length	Diameter	Height	Whole weight	Shucked weight	Viscera weight	Shell weight
0	M	0.645	0.475	0.155	1.2380	0.6185	0.3125	0.3005
1	M	0.580	0.460	0.160	0.9830	0.4785	0.2195	0.2750
2	M	0.560	0.420	0.140	0.8395	0.3525	0.1845	0.2405
3	M	0.570	0.490	0.145	0.8740	0.3525	0.1865	0.2350
4	I	0.415	0.325	0.110	0.3580	0.1575	0.0670	0.1050

💡 - 전처리 전
'id', 'Sex', 'Length', 'Diameter', 'Height', 'Whole weight', 'Whole weight.1', 'Whole weight.2', 'Shell weight', 'Rings'

- 전처리 후
'Sex', 'Length', 'Diameter', 'Height', 'Whole weight', '@ 'Shucked weight', 'Viscera weight', 'Shell weight', 'Rings'

추가 데이터 병합

```
[ ] 1 # train데이터와 original(abalone)데이터 병합
     2 train = pd.concat([train, original], axis = 0, ignore_index=True)
     3 train.head()
```

	Sex	Length	Diameter	Height	Whole weight	Shucked weight	Viscera weight	Shell weight	Rings
0	F	0.550	0.430	0.150	0.7715	0.3285	0.1465	0.2400	11
1	F	0.630	0.490	0.145	1.1300	0.4580	0.2765	0.3200	11
2	I	0.160	0.110	0.025	0.0210	0.0055	0.0030	0.0050	6
3	M	0.595	0.475	0.150	0.9145	0.3755	0.2055	0.2500	10
4	I	0.555	0.425	0.130	0.7820	0.3695	0.1600	0.1975	9

💡 - 전처리 전
train.shape = (90615, 9)
original.shape = (4177, 9)

- 전처리 후
train.shape = (94792, 9)



피처엔지니어링 및 전처리

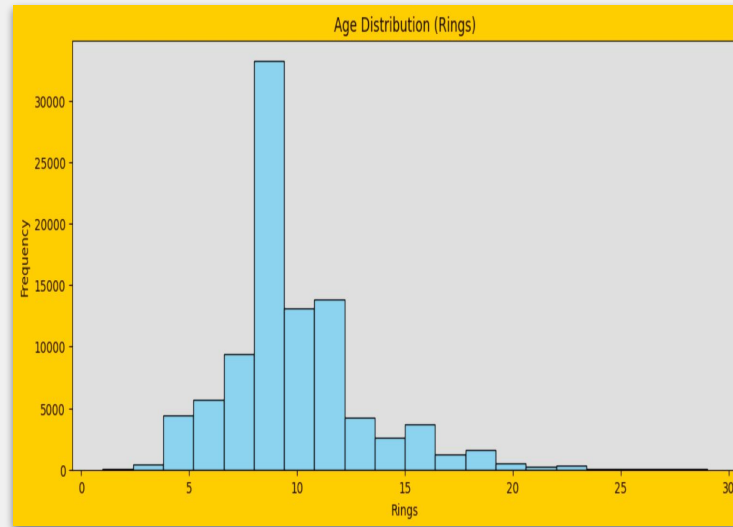
Feature Engineering & preprocessing

타겟 피쳐(Rings)에 대한 로그 변환

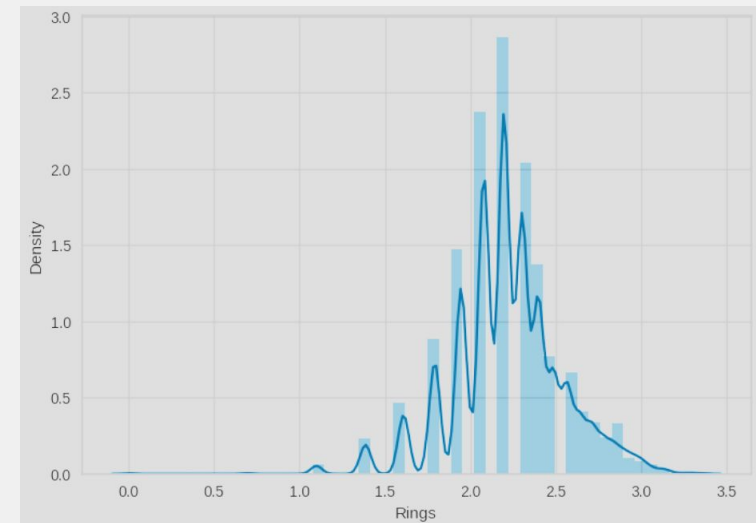
```
y = train['Rings']  
y_log = np.log(1+y)
```

```
[ ] 1 # 로그 변환 전  
     2 y.head()  
  
0    11  
1    11  
2     6  
3    10  
4     9  
     Name: Rings, dtype: int64  
  
[ ] 1 # 로그 변환 후  
     2 y_log.head()  
  
0    2.484907  
1    2.484907  
2    1.945910  
3    2.397895  
4    2.302585  
     Name: Rings, dtype: float64
```

EDA 단계의 타겟 피쳐(Rings) 분포



로그변환 후 타겟 피쳐(Rings) 분포



피처엔지니어링 및 전처리

Feature Engineering & preprocessing

성별 데이터 인코딩

```
1 encoder = OneHotEncoder(sparse_output = False, handle_unknown = 'ignore')
2
3 train = pd.concat([train.iloc[:,1:], pd.DataFrame(encoder.fit_transform(train[['Sex']]).astype('int'), columns = encoder.categories_[0]), axis = 1)
4 train.head()
```

	Length	Diameter	Height	Whole weight	Whole weight.1	Whole weight.2	Shell weight	F	I	M
0	0.550	0.430	0.150	0.7715	0.3285	0.1465	0.2400	1	0	0
1	0.630	0.490	0.145	1.1300	0.4580	0.2765	0.3200	1	0	0
2	0.160	0.110	0.025	0.0210	0.0055	0.0030	0.0050	0	1	0
3	0.595	0.475	0.150	0.9145	0.3755	0.2055	0.2500	0	0	1
4	0.555	0.425	0.130	0.7820	0.3695	0.1600	0.1975	0	1	0

steps:

[Generate code with train](#)

[View recommended plots](#)

```
1 test = pd.concat([test.iloc[:,1:], pd.DataFrame(encoder.transform(test[['Sex']]).astype('int'), columns = encoder.categories_[0]), axis = 1)
2 test.head()
```

	Length	Diameter	Height	Whole weight	Whole weight.1	Whole weight.2	Shell weight	F	I	M
0	0.645	0.475	0.155	1.2380	0.6185	0.3125	0.3005	0	0	1
1	0.580	0.460	0.160	0.9830	0.4785	0.2195	0.2750	0	0	1
2	0.560	0.420	0.140	0.8395	0.3525	0.1845	0.2405	0	0	1
3	0.570	0.490	0.145	0.8740	0.3525	0.1865	0.2350	0	0	1
4	0.415	0.325	0.110	0.3580	0.1575	0.0670	0.1050	0	1	0

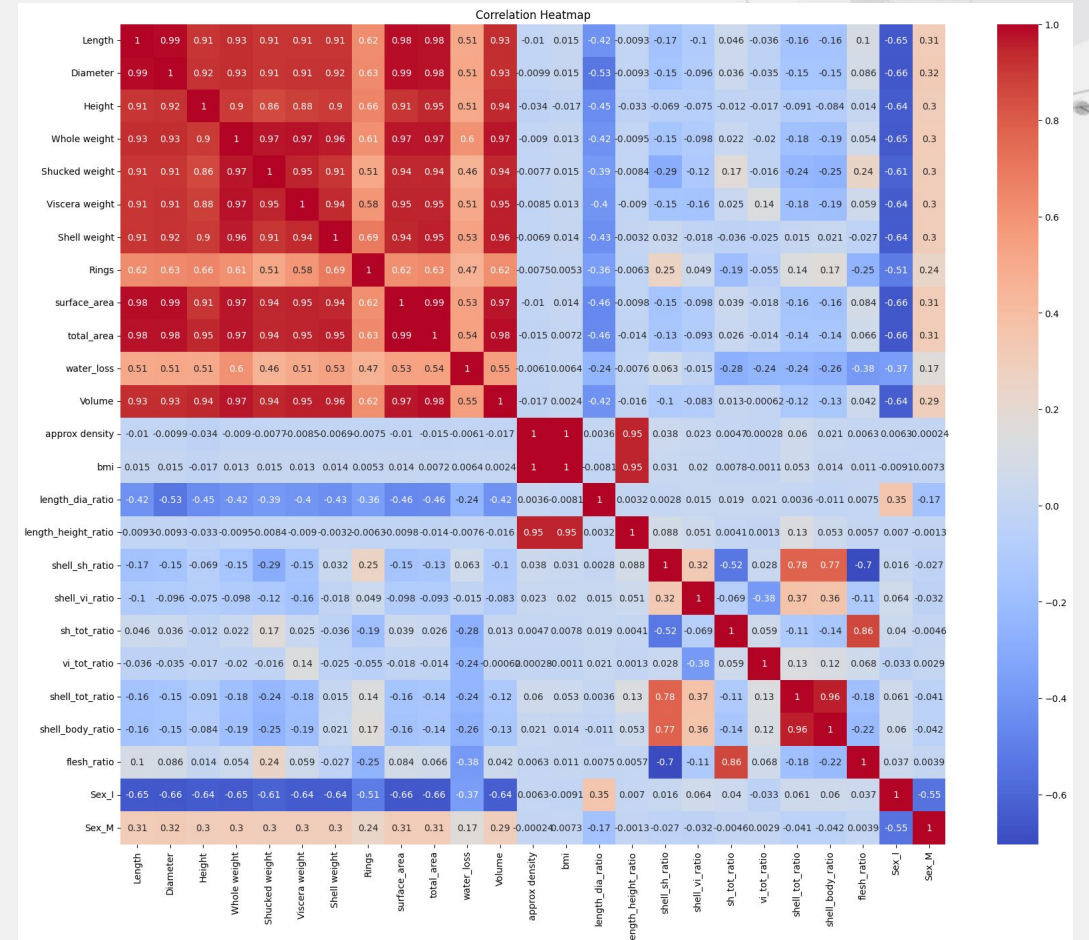
피처엔지니어링 및 전처리

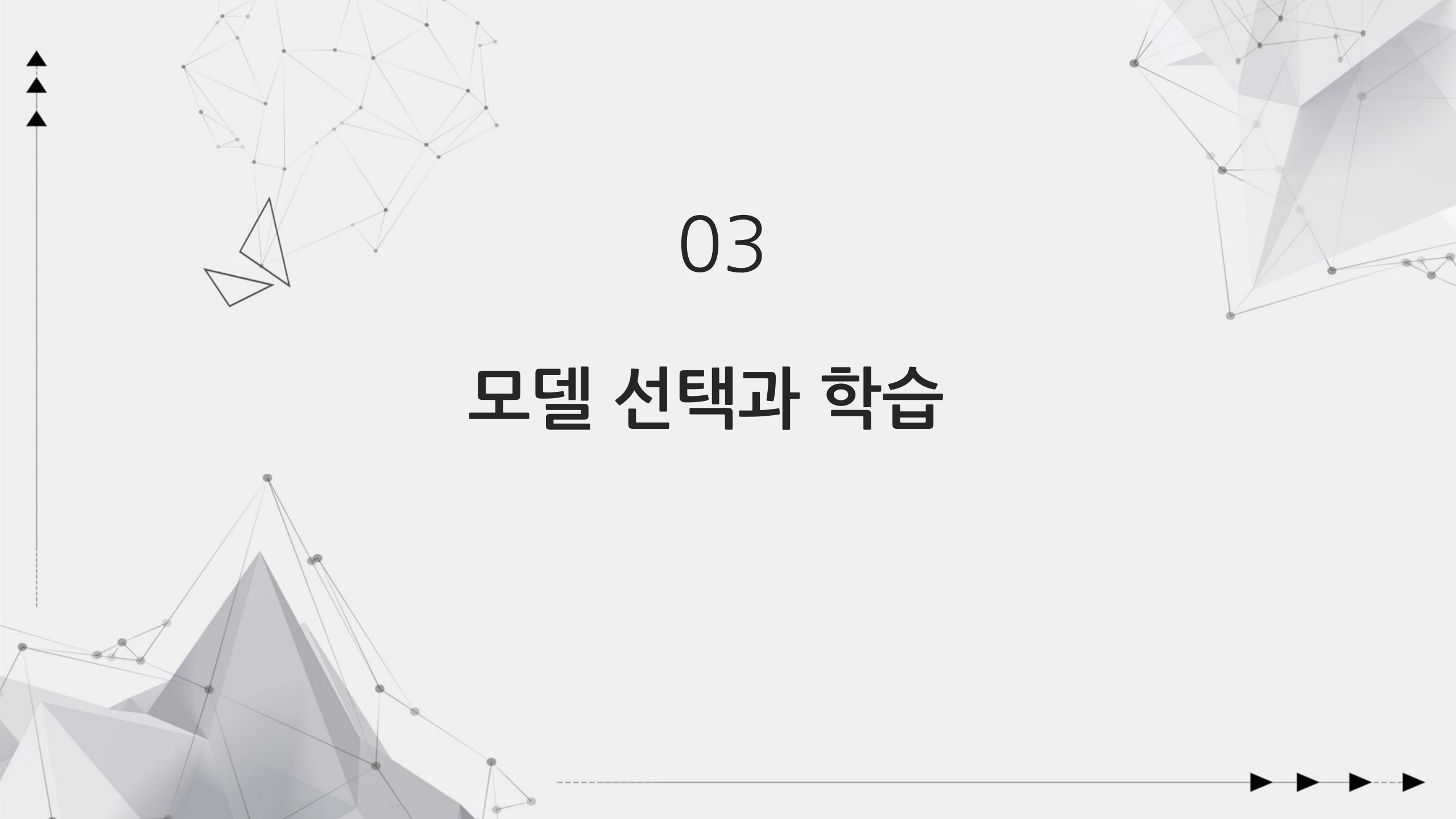
Feature Engineering & preprocessing

파생 변수 추가

```
1 def new_features(data):
2     df=data.copy()
3
4     # Clean the weights by capping the over weights with total body weights
5     df['Shell_weight']=np.where(df['Shell_weight']>df['Whole_weight'],df['Whole_weight'],df['Shell_weight'])
6     df['Viscera_weight']=np.where(df['Viscera_weight']>df['Whole_weight'],df['Whole_weight'],df['Viscera_weight'])
7     df['Shucked_weight']=np.where(df['Shucked_weight']>df['Whole_weight'],df['Whole_weight'],df['Shucked_weight'])
8
9     # Abalone Surface area
10    df['surface_area']=df['Length']*df['Diameter']
11    df['total_area']=2*(df['surface_area']+df['Height']*df['Diameter']+df['Length']*df['Height'])
12
13    # Abalone density approx
14    df['approx_density']=df['Whole_weight']/(df['surface_area']*df['Height']+1e-5)
15
16    # Abalone BMI
17    df['bmi']=df['Whole_weight']/(df['Height']**2+1e-5)
18
19    # Measurement derived
20    df['length_dia_ratio']=df['Length']/(df['Diameter']+1e-5)
21    df['length_height_ratio']=df['Length']/(df['Height']+1e-5)
22    df['shell_shuck_ratio']=df['Shell_weight']/(df['Shucked_weight']+1e-5)
23    df['shell_viscera_ratio']=df['Shell_weight']/(df['Viscera_weight']+1e-5)
24
25    df['viscera_tot_ratio']=df['Viscera_weight']/(df['Whole_weight']+1e-5)
26    df['shell_tot_ratio']=df['Shell_weight']/(df['Whole_weight']+1e-5)
27    df['shuck_tot_ratio']=df['Shucked_weight']/(df['Whole_weight']+1e-5)
28    df['shell_body_ratio']=df['Shell_weight']/(df['Shell_weight']+df['Whole_weight']+1e-5)
29    df['flesh_ratio']=df['Shucked_weight']/(df['Whole_weight']+df['Shucked_weight']+1e-5)
30
31    df['inv_viscera_tot']= df['Whole_weight'] / (df['Viscera_weight']+1e-5)
32    df['inv_shell_tot']= df['Whole_weight'] / (df['Shell_weight']+1e-5)
33    df['inv_shuck_tot']= df['Whole_weight'] / (df['Shucked_weight']+1e-5)
34
35    # Water Loss during experiment
36    df['water_loss']=df['Whole_weight']-df['Shucked_weight']-df['Viscera_weight']-df['Shell_weight']
37    df['water_loss']=np.where(df['water_loss']<0,min(df['Shucked_weight'].min(),df['Viscera_weight'].min(),df['Shell_weight'].min()),df['water_loss'])
38    return df
39
40
41 train=new_features(train)
42 test=new_features(test)
43 original=new_features(original)
```

파생 변수 추가 상관관계





03

모델 선택과 학습

모델 선택과 학습

교차검증

- Fold의 개수를 조정

다양한 모델 및 기법 사용

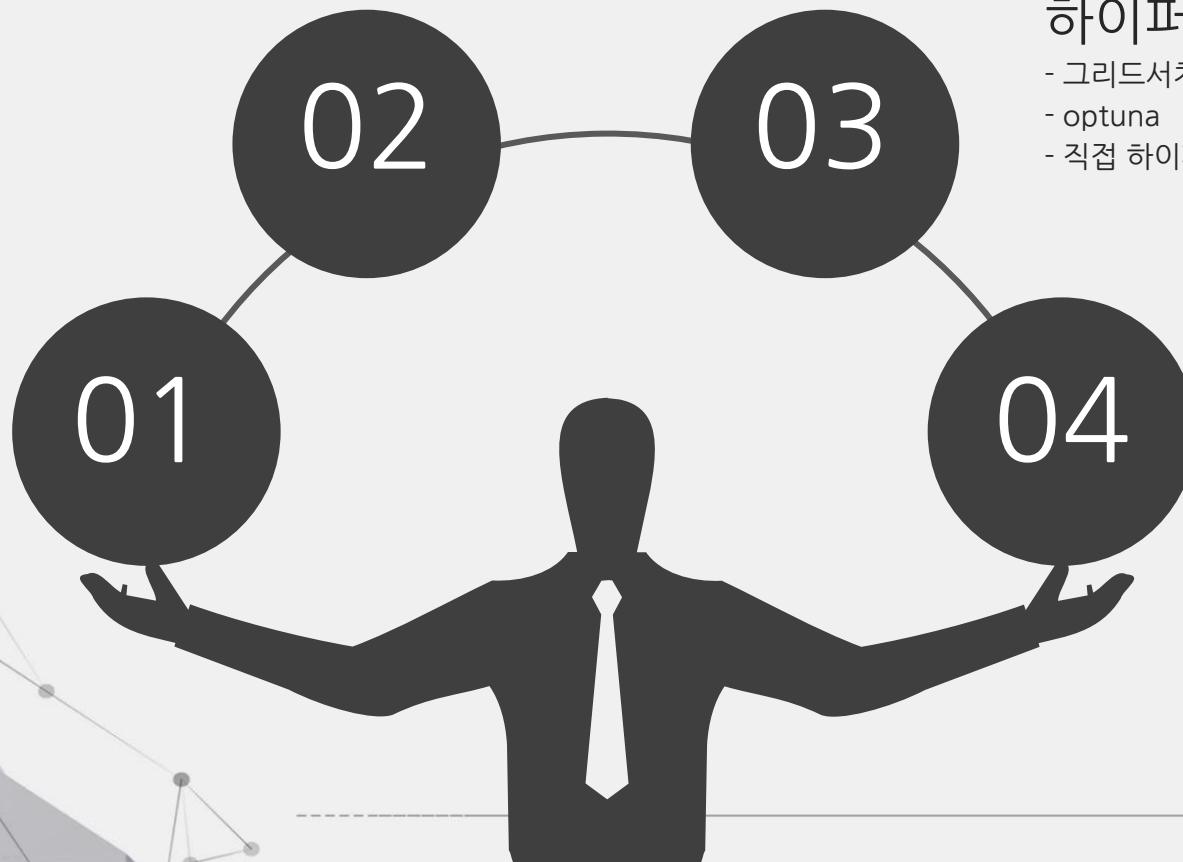
- LightGBM
- XGBoost
- CatBoost
- 앙상블(Blending, Voting)

하이퍼 파라미터 조정

- 그리드서치 cv
- optuna
- 직접 하이퍼 파라미터 수정

피처 엔지니어링

- 특정 피처에 대한 임계값을 설정
- 범주형 피처에 대한 인코딩
- 파생피처 생성



▲ 모델 선택과 학습 ▲ Model Selection and Training

 **LightGBM**

XGBoost

 **CatBoost**

Input Data

type	feature	count
categorical	Sex	1
continuous	Length, Diameter, Height, Whole weight, Shucked weight, Viscera weight, Shell weight, Rings	8

- 기존 피처만 이용해서 학습에 사용(새로운 피처 생성 후 성능 저하 확인)

▲모델 선택과 학습

▲Model Selection and Training

▲parameter

LightGBM

num_leaves	200
bagging_freq	7
boosting_type	"gbdt"
min_child_samples	91
objective	'regression'
learning_rate	0.0955
bagging_fraction	0.6502062728410578
feature_fraction	0.7058843944694884
device	'cpu'

XgBoost

max_depth	20
device	'cuda'
booster	'dart'
lambda	0.456836886068415
alpha	0.6422509164613671
subsample	0.9365423486036913
objective	'reg:squaredlogerror'
learning_rate	0.0955
colsample_bytree	0.8111849113860014

CatBoost

depth	20
max_bin	464
min_data_in_leaf	78
loss_function	RMSE
grow_policy	Lossguide
bootstrap_type	Bernoulli
subsample	0.83862137638162
l2_leaf_reg	8.365422739510098
random_strength	3.296124856352495
learning_rate	0.0955



성능평가와 결과분석

Performance Evaluation and Result Analysis

Blending

```
(0.14700)FM_I top3model(cat, lgbm, rf)
(0.14708)FM_I top4model(cat,lgbm,xgb,rf)
(0.14843)FM_I top4model_feature(피쳐 추가 ...)
```

```
[ ] 1 top_3_model_gender = compare_models(fold=5, round=3, n_select=3, errors='ignore')
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
catboost	CatBoost Regressor	0.128	0.030	0.173	0.537	0.053	0.054	9.280
lightgbm	Light Gradient Boosting Machine	0.128	0.030	0.174	0.535	0.053	0.054	0.518
rf	Random Forest Regressor	0.131	0.031	0.177	0.520	0.054	0.055	50.704
xgboost	Extreme Gradient Boosting	0.130	0.031	0.177	0.520	0.054	0.055	1.172



```
1 # 모델 블렌딩
2 reg_blended_FM = blend_models(estimator_list=top_3_model_gender, fold=10)
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	0.1279	0.0301	0.1734	0.5338	0.0527	0.0534
1	0.1290	0.0306	0.1750	0.5261	0.0531	0.0539
2	0.1297	0.0342	0.1850	0.5029	0.0614	0.0538
3	0.1300	0.0299	0.1730	0.5409	0.0512	0.0551
4	0.1249	0.0278	0.1667	0.5530	0.0485	0.0523
5	0.1263	0.0297	0.1722	0.5418	0.0529	0.0534
6	0.1267	0.0292	0.1708	0.5569	0.0510	0.0541
7	0.1270	0.0295	0.1717	0.5421	0.0529	0.0538
8	0.1245	0.0279	0.1669	0.5706	0.0518	0.0528
9	0.1285	0.0290	0.1702	0.5534	0.0495	0.0538
Mean	0.1274	0.0298	0.1725	0.5421	0.0525	0.0536
Std	0.0018	0.0017	0.0049	0.0177	0.0033	0.0007

상위 모델과 fold 값을 블렌딩을 사용하여 조절한 결과, Top3 모델인 cat, lgbm, rf를 사용한 10 폴드가 가장 좋은 성능

제출 결과, RMSLE가 0.14700으로 상위 25% 점수로 상위권 진입 실패
-> Voting으로 변경



성능평가와 결과분석

Performance Evaluation and Result Analysis

Voting

(0.14564) submission_voting_fold10(파라미터 수정1)
(0.14565) submission_voting_fold10(파라미터 수정3)
(0.14568) submission_voting_fold10(파라미터 수정2)
(0.14576) submission_voting_fold5(원본 파라미터)
(0.14606) submission_voting_feature(remains, volume)

Voting + fold change

(0.14559) submission_voting_fold11(최종)
(0.14559) submission_voting_fold13(최종)
(0.14559) submission_voting_fold14(최종)
(0.14560) submission_voting_fold12(최종)
(0.14563) submission_voting_fold15(최종)
(0.14564) submission_voting_fold10(최종)
(0.14565) submission_voting_fold9(최종)
(0.14567) submission_voting_fold7(최종)
(0.14567) submission_voting_fold8(최종)
(0.14569) submission_voting_fold6(최종)
(0.14576) submission_voting_fold5(최종)

Voting + ensemble

(0.14558) voting_fold10_top3(가중치_1, 1, 1)
(0.14559) voting_fold10_top5(가중치_1, 1, 1, 1, 1)
(0.14565) voting_fold10_top4(가중치_10,10,1,1)_ensemble
(0.14565) voting_fold10_top5(가중치 모두 1로 동일)_ensemble
(0.14566) voting_fold10_top4(가중치_10, 1, 1, 1)_ensemble
(0.14566) voting_fold10_top4(가중치_10, 10, 10, 1)_ensemble
(0.14566) voting_fold10_top7(가중치_10,9,8,7,6,5,4)_ensemble
(0.14567) voting_fold10_top3(가중치_10,1,1)_ensemble

- XGBoost, CatBoost, LGBM 사용
- 새로운 피쳐(remains, volume) 추가한 결과 → 성능 저하
- 원본과 파라미터, 10fold 기반 수정 진행
- 가장 좋은 성능인 RMSLE 결과 → 0.14564

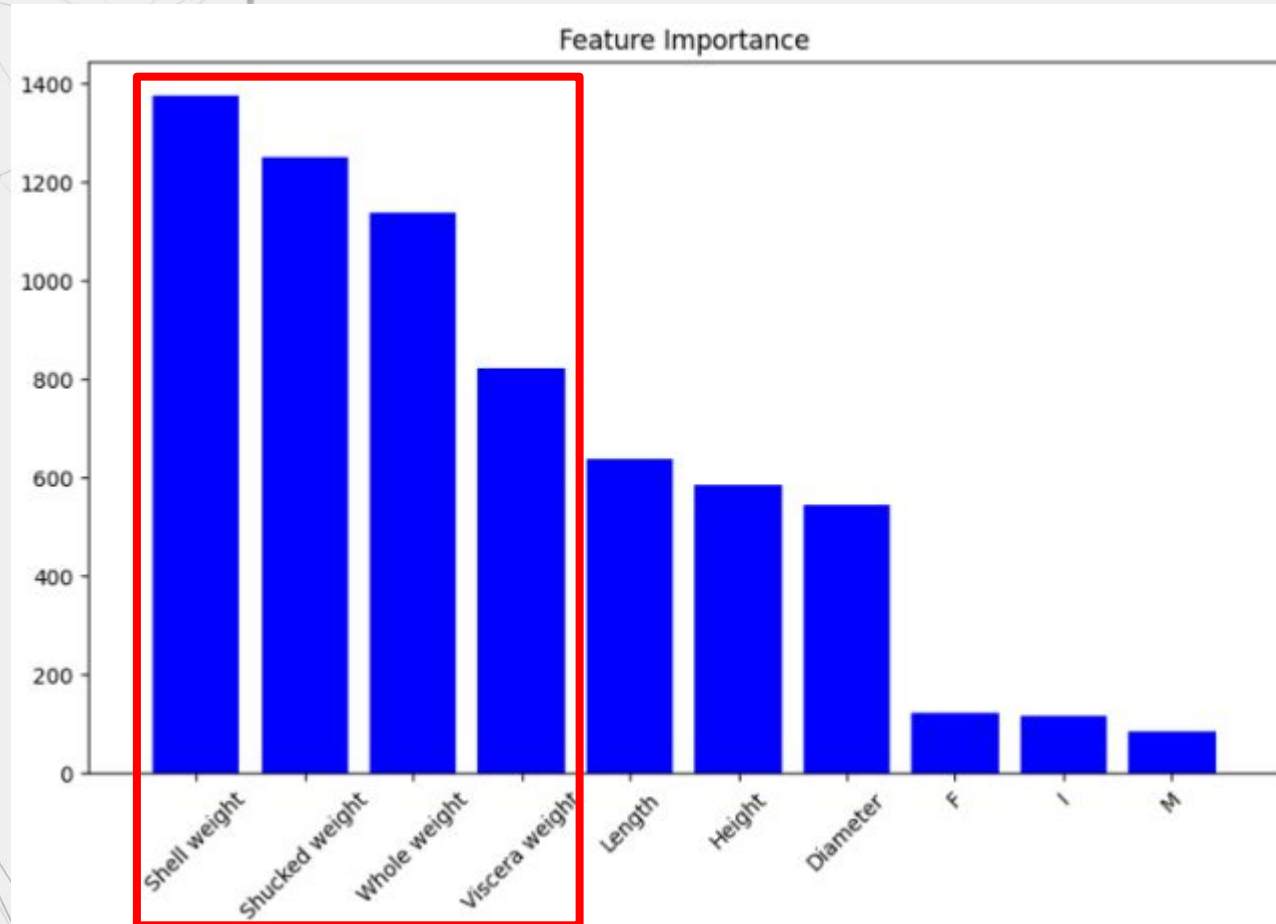
- 앞선 3개 모델 사용 유지
- fold 범위 → 5 ~ 15
- 점수 확인 결과 : fold(11, 13, 14) → (0.1455점대)

- fold(11, 13, 14) 선택
- 가중치 적용하여 앙상블(동일 가중치 적용)
- 결과 : 가장 높은 점수(0.14558)



▲ 성능평가와 결과분석

▲ Performance Evaluation and Result Analysis



무게 관련 피쳐 중요도가 높고, 그 뒤로 길이 관련 피쳐들이 뒤따르는 것을 확인



Kaggle 제출시 결과



vote_fold_top3(111).csv

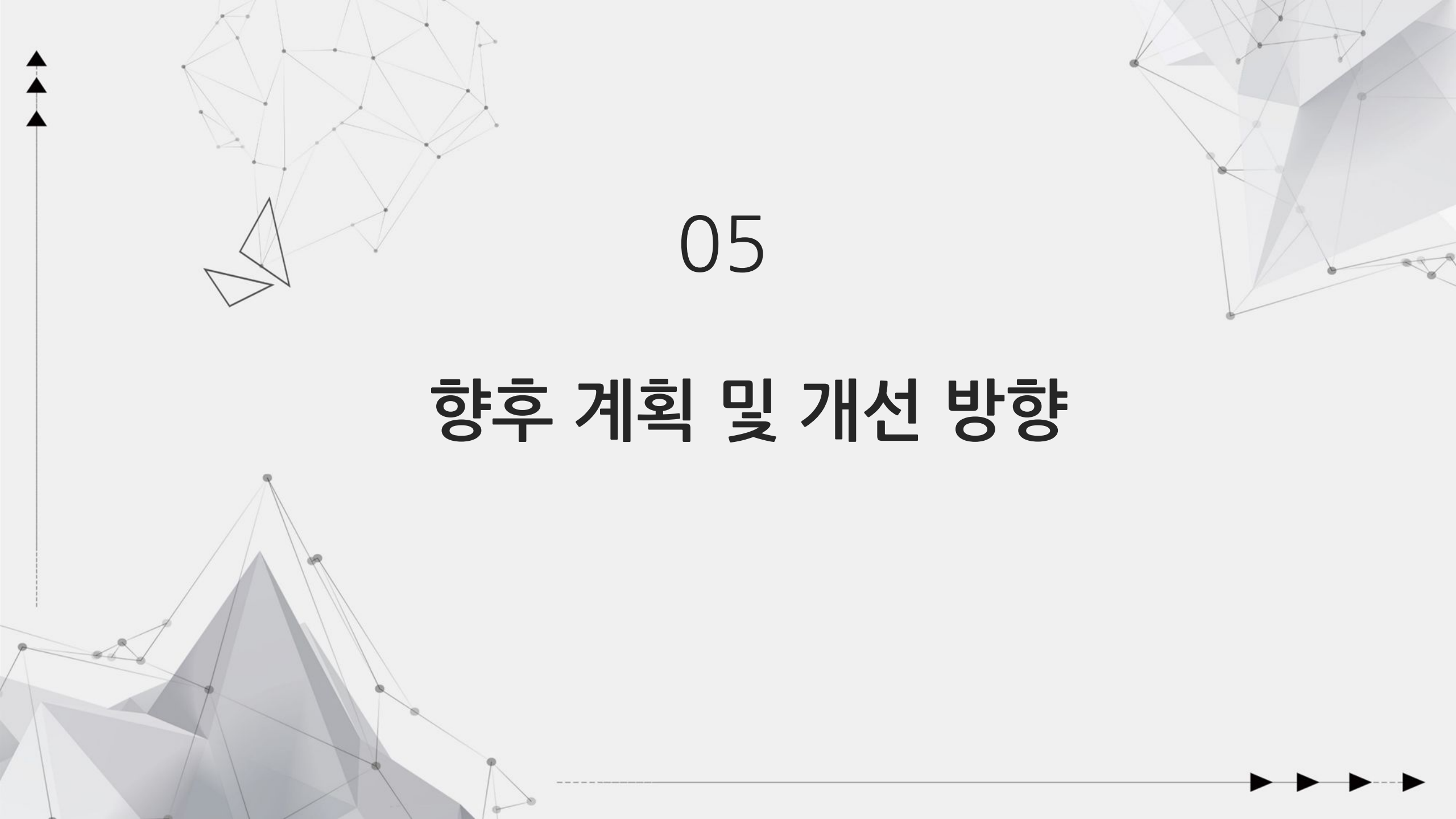
완료 · 5시간 전

0.14558

199	하디 가루디		0.14557	상	7일
200	전사		0.14557	11	5d
201	파벨 니콜라이체프		0.14557	1	12일
202	아흐메드 아불크헤어		0.14557	1	10일
203	크리스베밥		0.14557	53	1일
204	곽재우		0.14558	16	1일
최고의 출품작입니다! 가장 최근 제출하신 점수는 0.14558로, 이전 점수인 0.14559보다 향상된 수치입니다. 잘 했어!			이것을 트윗하세요		
205	시범 심		0.14558	1	14일
206	데이터루		0.14558	5	8일
207	조자성		0.14559	10	1일

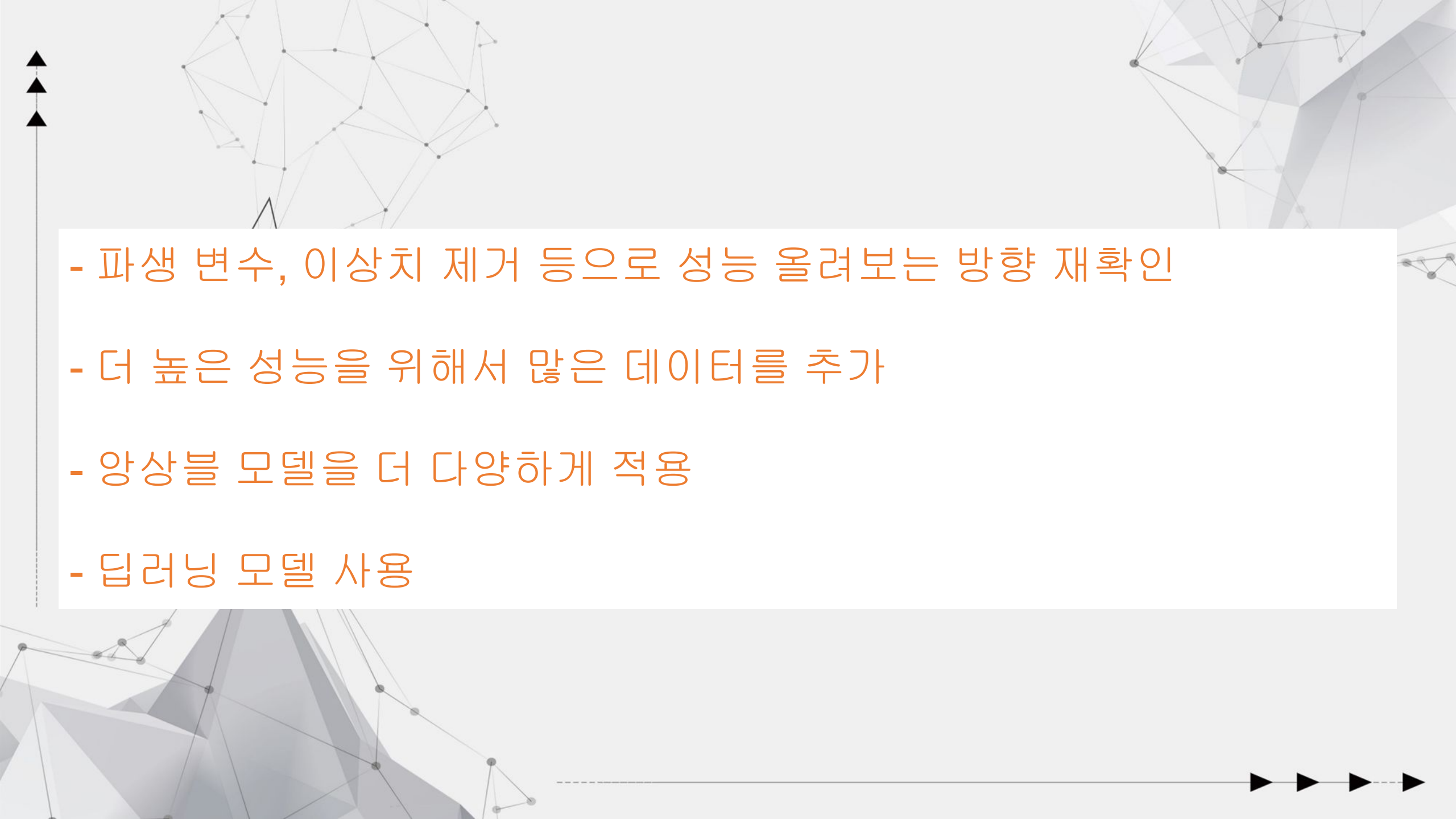
4월 25일 오후 1시 44분 기준 상위 10%(213등)안에 진입

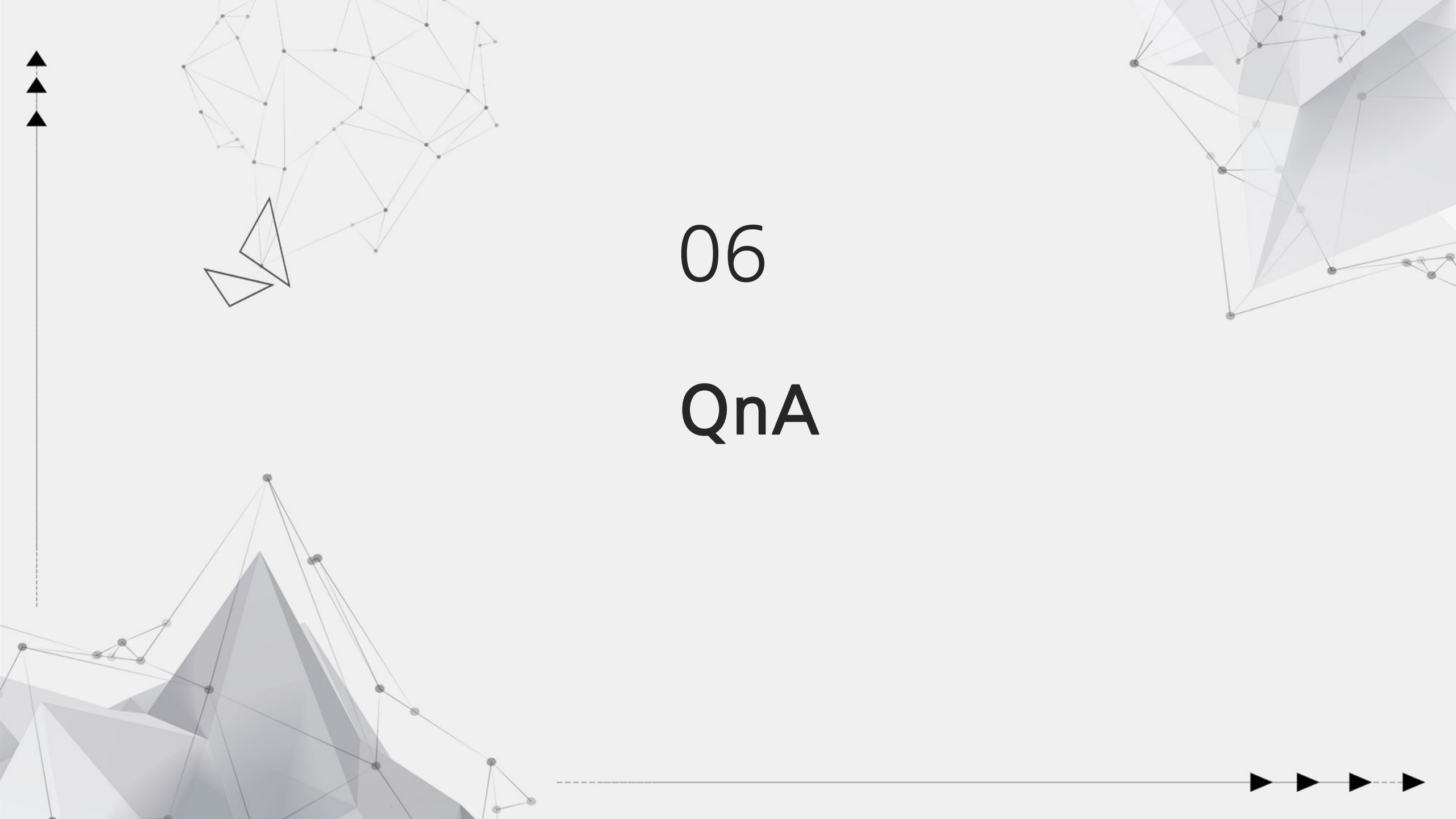
목표 했던 상위 10% 달성



05

향후 계획 및 개선 방향

- 
- 
- 파생 변수, 이상치 제거 등으로 성능 올려보는 방향 재확인
 - 더 높은 성능을 위해서 많은 데이터를 추가
 - 앙상블 모델을 더 다양하게 적용
 - 딥러닝 모델 사용
- 



06

QnA

출처

8-1. 참고자료

<https://github.com/Koda98/smoker-status-prediction/tree/main>

<https://www.kaggle.com/code/xxxxyyyy80008/smoker-status-prediction-lightgbm-baseline-no-fe>

<https://www.kaggle.com/code/arunklenin/ps3e24-smoking-cessation-prediction-binary>

<https://www.kaggle.com/code/arunklenin/ps4e4-abalone-age-prediction-regression/notebook#4.1-New-Features>

<https://www.kaggle.com/code/bunny11/voting-classifier/notebook>

<https://www.kaggle.com/code/satyaprakashshukl/cb-regression-analysis/notebook>

8-2. 출처

<https://www.kaggle.com/competitions/playground-series-s3e24/overview>

<https://www.kaggle.com/datasets/gauravduttakiit/smoker-status-prediction-using-biosignals>

<https://archive.ics.uci.edu/dataset/1/abalone>

<https://www.kaggle.com/competitions/playground-series-s4e4>



감사합니다